



Principles of Database Systems

By Xu Lizhen
School of Computer Science and Engineering, Southeast University
Nanjing, China



Main Contents

In this course, we will learn the basic concepts, principles and applications of database systems, especially the relational database systems. The contents mainly include :

- The data models, SQL language and user interfaces
- Key principles of DBMS (mainly architecture, query optimization, concurrency control, recovery, etc.)
- The security and integrity constraints of database
- Introduction of distributed database systems
- Some new research and application fields of database technology, such as data warehouse, data mining, XML data management, etc.

Principles of Database Systems, Xu Lizhen

2



References

- 1) Wang Nengbin, "Textbook of Database Systems"
- 2) Raghu Ramakrishnan, Johannes Gehrke, "Database Management Systems", 3rd Edition, McGraw-Hill Companies, 2002
- 3) Hector Garcia-Molina, Jeffrey.D.Ullman, "Database Systems: the Complete Book"
- 4) C.J.Date, "An Introduction to Database Systems"
- 5) Web Site of our course:
<http://cselab.seu.edu.cn/course/dbprinciple/>

Principles of Database Systems, Xu Lizhen

3



Table of Contents

1. **Introduction**
The history, classification, and main research contents of database systems; The database system; the concepts of data model
2. **Data Model***
Hierarchical and network model; Relational model; ER model; Object-Oriented model and other data models
3. **User Interfaces and SQL Language***
User interface; SQL language, including QL, DDL, DCL, DML, view, embedded SQL and dynamic SQL, etc.
4. **Database Management Systems***
The architecture of database systems, query optimization, file structure and index, transaction management, concurrency control, recovery mechanism

Principles of Database Systems, Xu Lizhen

4



Table of Contents

5. **The Security and Integrity Constraint**
The security model of database system; Integrity constraint and its expression, implementing method, assertion, trigger
6. **Database Design***
Design procedure; ER graph; Normalization of Relational Schema
7. **Distributed Database Systems**
What and Why DDBS, data distribution, distributed database design; Query optimization, distributed transaction management in DDBMS
8. **New Research and Application Fields**
Data warehouse, OLAP; Data mining; XML data management

Principles of Database Systems, Xu Lizhen

5



1. Introduction

What Is Database? What Is DBMS?

- A very large, integrated collection of data.
- Models real-world enterprise.
 - Entities (e.g., students, courses)
 - Relationships (e.g., electives)
- A Database Management System (DBMS) is a software package designed to store and manage databases.

Principles of Database Systems, Xu Lizhen

7

Files vs. Databases

- Application must stage large datasets between main memory and secondary storage (e.g., buffering, page-oriented access, 32-bit addressing, etc.)
- Special code for different queries
- Must protect data from inconsistency due to multiple concurrent users
- Crash recovery
- Security and access control

Principles of Database Systems, Xu Lizhen

8

A 文件本身没有数据模型，就是字符流

B 文件不能用来管理数据

Why Use a DBMS?

- Data independence and efficient access.
- Reduced application development time.
- Data integrity and security.
- Uniform data administration.
- Concurrent access, recovery from crashes.

Principles of Database Systems, Xu Lizhen

9

C 数据库支持语义丰富的数据模型，可更有效地管理数据

D 文件和数据库都可以管理数据

Why Study Databases ?

- Shift from computation to information
 - at the “low end”: scramble to webspace (a mess!)
 - at the “high end”: scientific applications
- Datasets increasing in diversity and volume.
 - Digital libraries, interactive video, Human Genome project, EOS project
 - ... need for DBMS exploding
- DBMS encompasses most of CS
 - OS, languages, theory, AI, multimedia, logic

Principles of Database Systems, Xu Lizhen

10

Data, Data Model and Data Schema

- Data are symbols for describing the things of real world. They are existing form of information.
- A data model is a collection of concepts and 辨析 definitions for describing data.
- A schema is a description of a particular collection of data, using a given data model.
- The relational model of data is the most widely used model today.
 - Main concept: relation, basically a table with rows and columns.
 - Every relation has a schema, which describes the columns, or fields.

A 数据是信息的表现形式

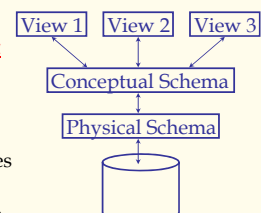
11

B 数据模型是表达现实世界信息的一组数据结构和规则

C 数据模式是某个具体数据库的数据结构的描述

Levels of Abstraction 考点

- Many views, single conceptual (logical) schema and physical schema.
 - Views describe how users see the data.
 - Conceptual schema defines logical structure
 - Physical schema describes the files and indexes used.



* Schemas are defined using DDL; data is modified/queried using DML.

Principles of Database Systems, Xu Lizhen

12

外模式：View 视图（虚表）逻辑上的一张表

数据独立性是依靠数据库的三级数据模式和它们之间的两级映射实现的



Example: University Database

- Conceptual schema:
 - Students(sid: string, name: string, login: string, age: integer, gpa: real)
 - Courses(cid: string, cname: string, credits: integer)
 - Enrolled(sid: string, cid: string, grade: integer)
- Physical schema:
 - Relations stored as unordered files.
 - Index on first column of Students.
- External Schema (View):
 - Course_info(cid: string, enrollment: integer)



Data Independence *

- Applications insulated from how data is structured and stored.
- **Logical data independence**: Protection from changes in logical structure of data.
- **Physical data independence**: Protection from changes in physical structure of data.

物理层面的变化不会导致逻辑层面使用的变化

* One of the most important benefits of using a DBMS!



The History of Database Technology and its Classification

(1) According to the development of data model

- No management (before 1960'): Scientific computing
- File system: Simple data management
- Demand of data management growing continuously, DBMS emerged.
 - 1964, the first DBMS (American): IDS, **network 网状模型**
 - 1969, the first commercial DBMS of IBM, **hierarchical 层次模型**
 - 1970, E.F.Codd (IBM) bring forward **relational data model 关系模型**
 - Other data model: Object Oriented, deductive, ER, ...



(2) According to the development of DBMS architectures

- Centralized database systems **集中式 大机器**
- Parallel database systems **并行处理**
- Distributed database systems (and Federated database systems) **分布式数据库 联邦数据库**
- Mobile database systems

(3) According to the development of architectures of application systems based on databases

- Centralized structure: Host + Terminal
- Distributed structure
- Client/Server structure
- Three tier / multi-tier structure
- Mobile computing
- Grid computing / Cloud computing

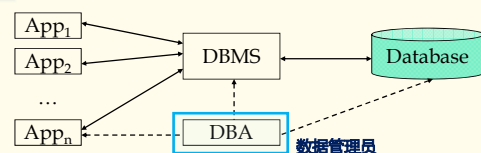


(4) According to the expanding of application fields

- OLTP **数据库 来做【联机事务处理】**
- Engineering Database
- Deductive Database
- Multimedia Database
- Temporal Database
- Spatial Database **【联机分析处理】**
- Data Warehouse, OLAP, Data Mining
- Knowledge Management
-



Database System

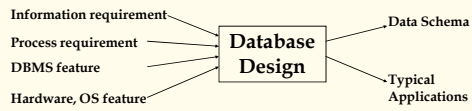


- Applications + DBMS + Database + DBA
- DBMS is the core of database system
 - High level user interfaces
 - Query processing and optimization
 - Catalog management
 - Concurrency control and Recovery
 - Integrity constraints checking
 - Access control



Life cycle of database systems

- Database system planning
- Database designing
- Database establishing and loading
- Database running, managing and maintaining
- Database extending and restructuring



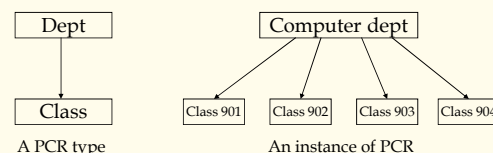
2. Data Model*

树形的结构来管理现实世界的事务

2.1 Hierarchical Data Model

Basic idea: because many things in real world are organized in hierarchy, hierarchical model manages to describe real world in a tree structure.

- Record and field 一个实体 -> 一个记录或多个域
- Parent-Child relationship (PCR): the most basic data relationship in hierarchical model. It expresses a 1:N relationship between two record types.



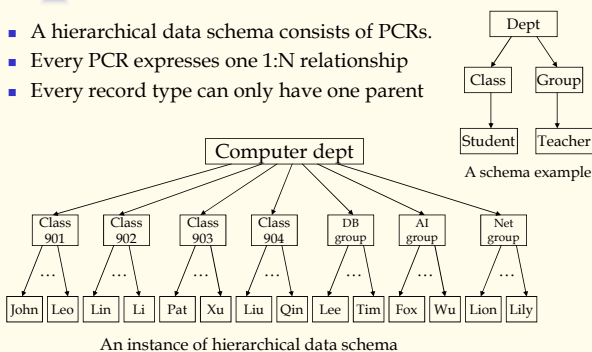
Principles of Database Systems, Xu Lizhen

2

一对多 比较容易实现 PCR

Hierarchical Data Schema

- A hierarchical data schema consists of PCRs.
- Every PCR expresses one 1:N relationship
- Every record type can only have one parent

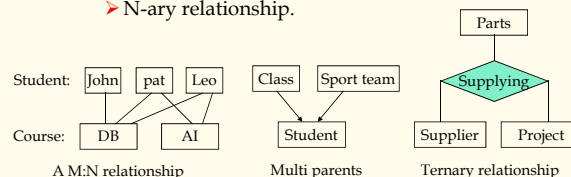


Principles of Database Systems, Xu Lizhen

3

Virtual Record

- In real world, many data are not hierarchical. It is hard to express them directly with PCR.
 - M:N relationship between different record types
 - A record type is the child of more than two PCRs.
 - N-ary relationship.



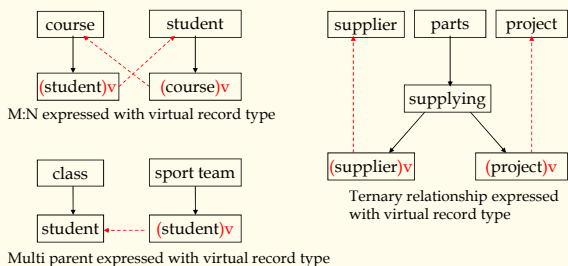
Principles of Database Systems, Xu Lizhen

4

多对多 如何在保持树形的情况下实现

Virtual Record

- To avoid redundant, virtual record is introduced to express above relationships. It is a pointer in fact.



Principles of Database Systems, Xu Lizhen

5

虚记录

通过让一个节点的子节点指向一个【虚记录】的方式实现多对多
【虚记录】类型并不真正存储数据，仅仅存储一个指向真实数据的【指针】
如图中的红色箭头

2.2 Network Data Model

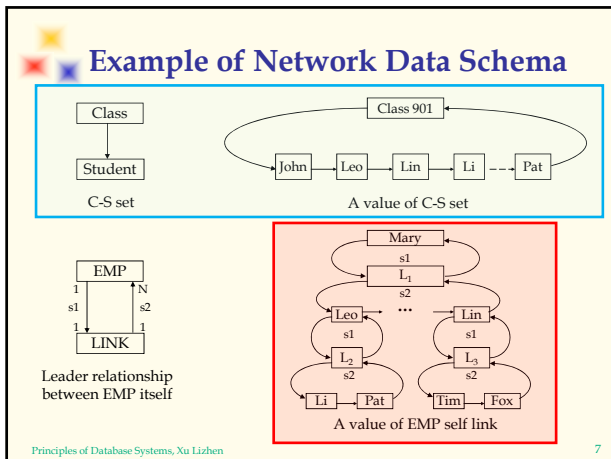
- The basic data structure is "set", it represent a 1:N relationship between things in real world. "1" side is called owner, and "N" side is called member. One record type can be the owner of multi sets, and also can be the member of multi sets. Many sets form a network structure to express real world.
- It breakthrough the limit of hierarchical structure, so can express non-hierarchical data more easy.
- Record and data items: data items are similar as field in hierarchical model, but it can be vector.
- Set: express the 1:N relationship between two record types.
- LINK record type: used to express self relationship, M:N relationship and N-ary relationship.

Principles of Database Systems, Xu Lizhen

6

基本数据结构【set】表示了1个【主记录】->N个【属记录】的关系

网状模型的基本数据结构
是链表

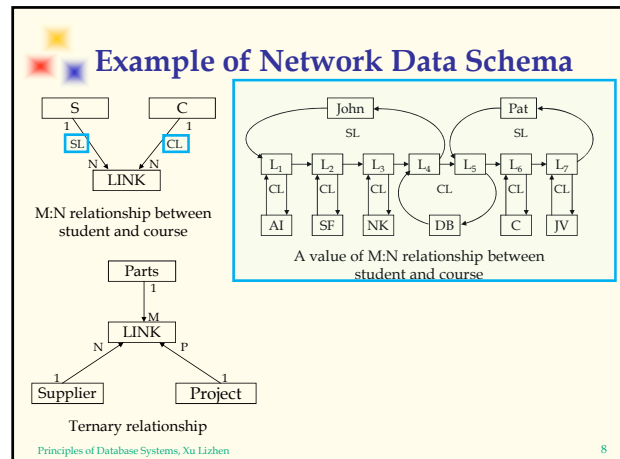


为了表达 employee 之间的关系，使用LINK记录类型 在EMP和LINK记录类型之间定义了一个1: 1的关系 如图中的s1

层次模型通过引入虚记录来表达记录型之间的多对多联系

网状模型通过引入LINK记录来表达记录型之间的多对多联系

虚记录和LINK记录的实现都涉及复杂的指针操作，对用户不友好



多对多 将两个记录都能链接到一组相同的LINK上实现多对多 (两组对称的1:N形成M:N)

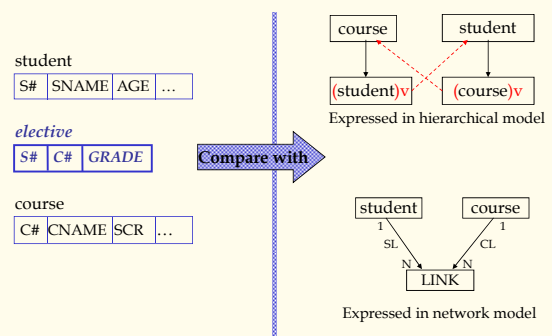
2.3 Relational Data Model

- The basic data structure is "table", or relation. The things and the relationships between them in real world are all expressed as tables, so it can be researched in strict mathematic methods. It raises the database technology to a theory height. Its features:
 - Based on set theory, high abstract level
 - Shield all lower details, simple and clear, easy to understand
 - Can establish new algebra system -- relational algebra
 - Non procedure query language -- SQL
 - Soft link -- the essential difference with former data models

Principles of Database Systems, Xu Lizhen

9

Understand Soft link



10

Attributes and Domain

- The features of an entity in real world are expressed as attributes in relational model. E.g. a student can be described with the attributes such as name, sid, gender, age, birthday, nationality, etc.
- The value scope of an attribute is called its domain.
 - Atomic data --- 1NF
 - Null

Principles of Database Systems, Xu Lizhen

11

Domain : 属性的值域
两个要求
1. 表是数据必须为基本数据类型是原子的 (1NF)
2. 可以允许为空值 (并不是空串) (NULL)

Relation and Tuple

- An entity of real world can be expressed as one or more than one relations.
- A relation is a N-ary relationship defined on all of its attribute domain. Suppose a relation R with attributes $A_1, A_2, \dots, A_n$, the corresponding domains are $D_1, D_2, \dots, D_n$, then R can be expressed as:

$$R = (A_1/D_1, A_2/D_2, \dots, A_n/D_n), \text{ or } R = (A_1, A_2, \dots, A_n)$$
- This is called the schema of R, and n is the number of attributes, called the degree of R. $A_i (1 \leq i \leq n)$ is attribute name.

Principles of Database Systems, Xu Lizhen

12

R [关系] = (A [属性值] / D [属性值域])



Relation and Tuple

- An instance (value) of R can be expressed as r or $r(R)$, it is a set of n-tuple:
 $r = \{t_1, t_2, \dots, t_m\}$
every tuple t can be expressed as:
 $t = \langle v_1, v_2, \dots, v_n \rangle, v_i \in D_i, 1 \leq i \leq n$
that is:
 $t \in D_1 \times D_2 \times \dots \times D_n, 1 \leq i \leq n$
that is:
 $r \subseteq D_1 \times D_2 \times \dots \times D_n, 1 \leq i \leq n$
- Relation is also called *table*. Attribute is also called *column*, and tuple is also called *row*.

Principles of Database Systems, Xu Lizhen

13

一个关系具体的实例是一个tuple组成的集合
一个tuple就是一行



Primary Key

- A set of attributes is a **candidate key** for a relation if :
 - No two distinct tuples can have same values in this set of attributes, and
 - This is not true for any subset of this set of attributes.
 - Part 2 false? A **superkey**, **candidate key** 的超集
 - If there's >1 key for a relation, one of the keys is chosen (by DBA) to be the **primary key**, and the others are called **alternate key**. 候选键中被选出来了的成为pk, 其他的ak
 - If the **primary key** consists of all attributes of a relation, it is called **all key**. 必须全选
- That means, the key can decide a tuple uniquely.
- E.g., *sid* is a key for Students. (What about *name*?)
The set {*sid*, *gpa*} is a superkey.

Principles of Database Systems, Xu Lizhen

14

一组属性集合可以成为【候选键】，当且仅当该集合为恰好使得在tuple中两两不同key可以是一个属性组 (a1, a2, a3)



Foreign Keys, Referential Integrity

- Foreign key** : Set of attributes in one relation that is used to 'refer' to a tuple in another relation. (Must correspond to primary key of the second relation.) Like a 'logical pointer'.
- E.g. *sid* is a foreign key referring to **Students**:
 - Enrolled(*sid*: string, *cid*: string, *grade*: string)
 - If all foreign key constraints are enforced, **referential integrity** is achieved, i.e., no dangling references. 引用完整性
 - Have you forgotten **soft link**?

Principles of Database Systems, Xu Lizhen

15



An Example of Referential Integrity

- Only students listed in the Students relation should be allowed to enroll for courses.

Enrolled

sid	cid	grade
53666	Carnatic101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Students

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

Principles of Database Systems, Xu Lizhen

16



Other Integrity Constraints

- Domain integrity constraint** 域完整性
 - An attribute's value must be a value **in the domain** of this attribute. This is the most basic constraint. All popular RDBMS are able to check domain integrity constraint automatically.
- Entity integrity constraint** 实体完整性
 - Every relation **should have a primary key**. The value of primary key of each tuple must be **unique**. Primary key **cannot be NULL**. This is so-called entity integrity constraint.

Principles of Database Systems, Xu Lizhen

17



Example Instances

- "Sailors", "Reserves" and "Boats" relations for our examples.

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

此处是R1为全键

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

B1

bid	bname	color
101	tiger	red
103	lion	green
105	hero	blue

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Principles of Database Systems, Xu Lizhen

18

Relational Algebra

- Basic operations:
 - > **Selection** (σ) Selects a subset of rows from relation.
 - > **Projection** (π) Deletes unwanted columns from relation.
 - > **Cross-product** ($\times$) Allows us to combine two relations.
 - > **Set-difference** ($-$) Tuples in reln. 1, but not in reln. 2.
 - > **Union** ($\cup$) Tuples in reln. 1 and in reln. 2.
- $\{\sigma, \pi, \cup, -, \times\}$ is a **complete operation set**. Any other relational algebra operations can be derived from them.
- Additional operations:
 - > Intersection, **join**, division, outer join, outer union: Not essential, but (very!) useful.
- Since each operation returns a relation, **operations can be composed!** (Algebra is "closed".)

Principles of Database Systems, Xu Lizhen

19

Projection

- Deletes attributes that are not in **projection list**.
- Schema of result** contains exactly the fields in the projection list, with the same names that they had in the input relation.
- Projection operator has to eliminate **duplicates!** (Why??)
 - > Note: real systems typically don't do duplicate elimination unless the user explicitly asks for it. (Why not?)

sname	rating
yuppy	9
lubber	8
guppy	5
rusty	10

$\pi_{sname, rating}(S2)$

age
35.0
55.5

$\pi_{age}(S2)$

Principles of Database Systems, Xu Lizhen

20

Selection

- Selects rows that satisfy **selection condition**.
- No duplicates in result! (Why?)
- Schema** of result identical to that of input relation.
- Result relation can be the **input** for another relational algebra operation! (Operator composition.)

sid	sname	rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$\sigma_{rating > 8}(S2)$

sname	rating
yuppy	9
rusty	10

$\pi_{sname, rating}(\sigma_{rating > 8}(S2))$

Principles of Database Systems, Xu Lizhen

21

Union, Intersection, Set-Difference

- All of these operations take two input relations, which must be **union-compatible**:
 - > Same number of fields.
 - > Corresponding attributes have the same type.
- What is the **schema** of result?

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

$S1 \cup S2$

sid	sname	rating	age
22	dustin	7	45.0

$S1 - S2$

sid	sname	rating	age
31	lubber	8	55.5
58	rusty	10	35.0

$S1 \cap S2$

Principles of Database Systems, Xu Lizhen

22

Cross-Product

- Each row of S1 is paired with each row of R1.
- Result schema** has one attribute per attribute of S1 and R1, with attribute names **inherited** if possible.
 - > **Conflict**: Both S1 and R1 have an attribute called *sid*.

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

- Renaming operator**: $\rho(C(1 \rightarrow sid1, 5 \rightarrow sid2), S1 \times R1)$

Principles of Database Systems, Xu Lizhen

23

Joins

- Condition Join**: $R \bowtie_C S = \sigma_C(R \times S)$

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	58	103	11/12/96

$S1 \bowtie_{S1.sid < R1.sid} R1$

- Result schema** same as that of cross-product.
- Fewer tuples than cross-product, might be able to compute more efficiently
- Sometimes called a **theta-join**.

Principles of Database Systems, Xu Lizhen

24

Joins

- **Equi-Join**: A special case of condition join where the condition c contains only *equalities*.

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

$$S1 \bowtie_{sid} R1$$

- **Result schema** similar to cross-product, but only one copy of attributes for which equality is specified.
- **Natural Join**: Equi-join on all **common attributes**.

Division

- Not supported as a primitive operator, but useful for expressing queries like:
 - Find sailors who have reserved **all** boats.
- Let A have 2 fields, x and y ; B have only field y :
 - $A/B = \{ \langle x \rangle \mid \exists \langle x, y \rangle \in A \ \forall \langle y \rangle \in B \}$
 - i.e., **A/B contains all x tuples (sailors) such that for every y tuple (boat) in B , there is an xy tuple in A .**
 - Or: If the set of y values (boats) associated with an x value (sailor) in A contains all y values in B , the x value is in A/B .
- In general, x and y can be any lists of fields; y is the list of fields in B , and $x \cup y$ is the list of fields of A .

Examples of Division A/B

sno	pno	pno	pno	pno
s1	p1	p2	p2	p1
s1	p2		p4	p2
s1	p3			p4
s1	p4			
s2	p1			
s2	p2			
s3	p2			
s4	p2			
s4	p4			

A

$A/B1$

$A/B2$

$A/B3$

Expressing A/B Using Basic Operators

- Division is not essential op; just a useful shorthand.
 - (Also true of joins, but joins are so common that systems implement joins specially.)
- **Idea**: For A/B , compute all x values that are not *disqualified* by some y value in B .
 - x value is *disqualified* if by attaching y value from B , we obtain an xy tuple that is not in A .

Disqualified x values: $\pi_x((\pi_x(A) \times B) - A)$

A/B : $\pi_x(A) - \text{all disqualified tuples}$

Outer Joins

- The extension of join operation. In join operation, only matching tuples fulfilling join conditions are left in results. Outer joins will keep unmated tuples, the vacant part is set **Null**:
 - Left outer join ($* \bowtie$)
 - Keep all tuples of left relation in the result.
 - Right outer join ($\bowtie *$)
 - Keep all tuples of right relation in the result.
 - Full outer join ($* \bowtie *$)
 - Keep all tuples of left and right relations in the result.

Examples of Outer Joins

S1	sid	sname	rating	age	R1	sid	bid	day
	22	dustin	7	45.0		22	101	10/10/96
	31	lubber	8	55.5		58	103	11/12/96
	58	rusty	10	35.0				

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96
31	Lubber	8	55.5	null	null

$S1 * \bowtie R1$

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

$S1 \bowtie * R1 =$
 $S1 \bowtie R1$ (Why?)

sid	sname	rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96
31	Lubber	8	55.5	null	null

$S1 * \bowtie * R1$

(3) $r * \bowtie_{C=D} s = \{ \{ \langle ABCDEF \rangle \} \mid (\{ \langle ABC \rangle \in r \wedge \{ \langle DEF \rangle \in s \wedge t.C=t.D \}) \vee (\{ \langle ABC \rangle \in r \wedge \neg(t.C \in s[D]) \wedge t.D=NULL \wedge t.E=NULL \wedge t.F=NULL) \}$



Outer Unions 外并

- The extension of union operation. It can union two relations which are not *union-compatible*.
- The attribute set in result is the union of attribute sets of two operands.
- The values of attributes which don't exist in original tuples are filled as NULL

sid	sname	rating	age	bid	day
22	dustin	7	45.0	null	null
31	Lubber	8	55.5	null	null
58	rusty	10	35.0	null	null
22	null	null	null	101	10/10/96
58	null	null	null	103	11/12/96

S1 $\cup$ R1

结果的元组数是参与运算的两个表元组数的和

31



Relational Calculus

- Relational Algebra needs to specify the order of operations; while relational calculus only needs to indicate the logic condition the result must be fulfilled.
- Comes in two flavors: *Tuple relational calculus (TRC)* and *Domain relational calculus (DRC)*.
- Calculus has *variables*, *constants*, *comparison ops*, *logical connectives* and *quantifiers*.
 - > *TRC*: Variables range over (i.e., get bound to) *tuples*.
 - > *DRC*: Variables range over *domain elements* (attribute values).
 - > Both TRC and DRC are simple subsets of first-order logic.
- Expressions in the calculus are called *formulas*. An answer tuple is essentially an assignment of constants to variables that make the formula evaluate to *true*.

Principles of Database Systems, Xu Lizhen

32

它可以将两个不满足并兼容条件的表做集合合并



Domain Relational Calculus

- Query* has the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n, x_{n+1}, \dots, x_{n+m}) \}$$
- $x_1, x_2, \dots, x_n, x_{n+1}, \dots, x_{n+m}$ are called *domain variables*. $x_1, x_2, \dots, x_n$ appear in result.
- > *Answer* includes all tuples $\langle x_1, x_2, \dots, x_n \rangle$ that make the formula $P(x_1, x_2, \dots, x_n, x_{n+1}, \dots, x_{n+m})$ be *true*.
- > *Formula* is recursively defined, starting with simple *atomic formulas* (getting tuples from relations or making comparisons of values), and building bigger and better formulas using the *logical connectives*.

Principles of Database Systems, Xu Lizhen

33



DRC Formulas

- Atomic formula*:
 - > $\langle x_1, x_2, \dots, x_n \rangle \in Rname$, or $X op Y$, or $X op constant$
 - > op is one of $<, >, =, \leq, \geq, \neq$
- Formula*:
 - > an atomic formula, or
 - > $\neg p, p \wedge q, p \vee q$, where p and q are formulas, or
 - > $\exists X(p(X))$, where variable X is *free* in $p(X)$, or
 - > $\forall X(p(X))$, where variable X is *free* in $p(X)$

Principles of Database Systems, Xu Lizhen

34



Free and Bound Variables

- The use of *quantifiers* $\exists X$ and $\forall X$ is said to *bind* X .
 - > A variable that is *not bound* is *free*.
- Let us revisit the definition of a *query*:

$$\langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n, x_{n+1}, \dots, x_{n+m})$$
- There is an important restriction: the variables $x_1, x_2, \dots, x_n$ that appear to the left of $\mid$ must be the *only* free variables in the formula $p(\dots)$.

Principles of Database Systems, Xu Lizhen

35



Find all sailors with a rating above 7

- $\{ \langle I, N, T, A \rangle \mid \langle I, N, T, A \rangle \in Sailors \wedge T > 7 \}$
- The condition $\langle I, N, T, A \rangle \in Sailors$ ensures that the domain variables I, N, T and A are bound to fields of the same Sailors tuple.
- The term $\langle I, N, T, A \rangle$ to the left of $\mid$ (which should be read as *such that*) says that every tuple $\langle I, N, T, A \rangle$ that satisfies $T > 7$ is in the answer.
- Modify this query to answer:
 - > Find sailors who are older than 18 or have a rating under 9, and are called 'Joe'.

Principles of Database Systems, Xu Lizhen

36

关系演算可能会出现不安全的查询

关系演算和关系代数的表达能力是等价的

它们是非过程化的关系运算表达方法

域关系演算的变量定义在属性上

元组关系演算的变量定义在元组上

Unsafe Queries, Expressive Power

- It is possible to write syntactically correct calculus queries that have an infinite number of answers! Such queries are called *unsafe*.
 - e.g., $\{S \mid \neg(S \in \text{Sailors})\}$
- It is known that every query that can be expressed in relational algebra can be expressed as a safe query in DRC / TRC; the converse is also true.
- Relational Completeness:** $\{\sigma, \pi, \cup, -, \times\}$ is a complete operation set. Relational calculus can express these five operations easily, so relational calculus is also *Relational Completeness*. SQL language is based on relational calculus, so it can express any query that is expressible in relational algebra / calculus.

Principles of Database Systems, Xu Lizhen

37

Tuple Relational Calculus

- Query** has the form:
 $\{t[\langle \text{attribute list} \rangle] \mid P(t)\}$
- t is called *tuple variable*.
- Answer** includes all tuples $t[\langle \text{attribute list} \rangle]$ that make the formula $P(t)$ be true.
- Example query: Find all sailors' name whose rating above 7 and younger than 50;
 $\{t[N] \mid t \in \text{Sailors} \wedge t.T > 7 \wedge t.A < 50\}$

Principles of Database Systems, Xu Lizhen

38

Remarks to Traditional Data Model

- Hierarchical, Network, and Relational Model
- Suitable for OLTP applications
- Based on record, can't orient to users or applications better
- Can't express the relationships between entities in a natural mode.
- Lack of semantic information
- Few data type, hard to fulfill the requirements of applications

Principles of Database Systems, Xu Lizhen

39

2.4 ER Data Model

- Entity:** Real-world object distinguishable from other objects. An entity is described (in DB) using a set of *attributes*.
- Entity Set:** A collection of similar entities. E.g., all employees.
 - All entities in an entity set have the same set of attributes. (Until we consider ISA hierarchies, anyway!)
 - Each entity set has a *key*.
 - Each attribute has a *domain*.
 - Permit combined or multi-valued attribute

Principles of Database Systems, Xu Lizhen

40

Relationship

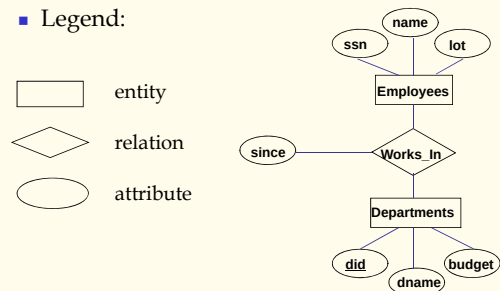
- Relationship:** Association among two or more entities. E.g., Attishoo works in Pharmacy department.
 - Relationship can have attributes
- Relationship Set:** Collection of similar relationships.
 - An n-ary relationship set R relates n entity sets $E_1 \dots E_n$; each relationship in R involves entities $e_1, \dots, e_n$
 - Same entity set could participate in different relationship sets, or in different "roles" in same set.

Principles of Database Systems, Xu Lizhen

41

ER Diagram

- Concept model: entity – relationship, be independent of practical DBMS.
- Legend:



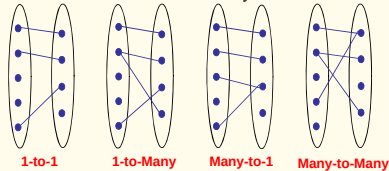
Principles of Database Systems, Xu Lizhen

42



Cardinality Ratio Constraints

- Relationships can be distinguished as 1:1, 1:N, and M:N. This is called cardinality ratio constraints.



- For example: an employee can work in many departments; a dept can have many employees. This M:N. In contrast, each dept has at most one manager and one employee can only be manager of one dept, then this is 1:1.



Participation Constraints

- We can further specify the minimal and max number an entity participates a relationship. This is called participation constraints.
- If a department must have a manager, then we say the Departments is *total participation* in Manages relationship (*vs. partial*). The minimal participating degree of Departments is 1.
- Another example: in the selected course relationship between Students and Courses, if we specify every student must select at least 3 courses and at most 6 courses, the participating degree of Students is said to be (3,6).



Advanced Topics of ER Model

- Weak entity
- Specialization and Generalization
 - Similar as inheriting in Object-Oriented data model
- Aggregation
 - Allows us to treat a relationship set as an entity set for purposes of participation in (other) relationships
- Category
 - Allow us to express an entity set consists of different types of entities. That is, a hybrid entity set.



2.5 Object-Oriented Data Model

- The shortage of relational data model
- Break through 1NF
- Object-Oriented analysis and programming
- Requirement of objects' permanent store
- Object-Relation DBMS
- Native (pure) Object-Oriented DBMS



2.6 Other Data Models

- Logic-based data model (Deductive DBMS)
 - Extend the query function of DBMS (especially recursive query function)
 - Promote the deductive ability of DBMS
- Temporal data model
- Spatial data model
- XML data model
 - Store data on internet
 - Common data exchange standard
 - Information systems integration
 - Expression of semi-structured data
 - ...
- Others



2.7 Summary

- Data model is the core of a DBMS
- A data model is a methodology to simulate real world in database
- In fact, every kind of DBMS has implemented a data model
- ☹ If there will be a data model which can substitute relational model and become popular data model, just as relational model substituted hierarchical and network model 30 years ago ???

3. User Interfaces and SQL Language*

User interface of DBMS

- A DBMS must offer some interfaces to support user to access database, including:
 - Query Languages
 - Interface and maintaining tools (GUI)
 - APIs
 - Class Library
- Query Languages
 - Formal Query Language
 - Tabular Query Language
 - Graphic Query Language
 - Limited Natural Language Query Language

Principles of Database Systems, Xu Lizhen

2

Example of TQL & GQL

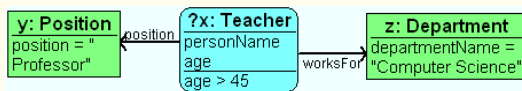
Find the names of all students in the department of Info. Science

Student	Sno	Sname	Ssex	Sage	Sdept
		P.T			IS

操作符，表示打印 (print)
实际是显示

示例元素，域变量

条件



Find all Teachers, which have position="Professor" and which have age>"45" and which work for department="Computer Science"

Principles of Database Systems, Xu Lizhen

3

Relational Query Languages

- Query languages: Allow manipulation and retrieval of data from a database.
- Relational model supports simple, powerful QLs:
 - Strong formal foundation based on logic.
 - Allows for much optimization.
- Query Languages != programming languages!
 - QLs not expected to be "Turing complete".
 - QLs not intended to be used for complex calculations.
 - QLs support easy, efficient access to large data sets.

Principles of Database Systems, Xu Lizhen

4

SQL语言的数学基础是：元组关系演算

Formal Relational Query Languages

- Two mathematical Query Languages form the basis for "real" languages (e.g. SQL), and for implementation:
 - *Relational Algebra*: More **operational**, very useful for representing execution plans.
 - *Relational Calculus*: Lets users describe what they want, rather than how to compute it. (**Non-operational, declarative**.)
- The most successful relational database language --- SQL (Structured Query Language, Standard Query Language(1986))

Principles of Database Systems, Xu Lizhen

5

SQL Language

- It can be divided into four parts according to functions.
 - Data Definition Language (DDL), used to define, delete, or alter data schema.
 - Query Language (QL), used to retrieve data
 - Data Manipulation Language (DML), used to insert, delete, or update data.
 - Data Control Language (DCL), used to control user's access authority to data.
- QL and DML are introduced in detail in this chapter.

Principles of Database Systems, Xu Lizhen

6

Important terms and concepts

- Base table 就是表, 表示一个关系
- View 视图, 虚表
- Data type supported 可支持数据类型
- NULL
- UNIQUE 属性是否允许重复值
- DEFAULT
- PRIMARY KEY 主键
- FOREIGN KEY 外键 另外一张表的主键
- CHECK (Integration Constraint)
为表定义完整性约束诸如年龄必须大于15岁

Principles of Database Systems, Xu Lizhen

7

Example Instances

- We will use these instances of the Sailors, Reserves and Boats relations in our examples.

R1

sid	bid	day
22	101	10/10/96
58	103	11/12/96

B1

bid	bname	color
101	tiger	red
103	lion	green
105	hero	blue

S1

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

S2

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

Principles of Database Systems, Xu Lizhen

8

Basic SQL Query

```
SELECT [DISTINCT] target-list
FROM relation-list
WHERE qualification
```

- relation-list** A list of relation names (possibly with a *range-variable* after each name).
- target-list** A list of attributes of relations in *relation-list*
- qualification** Comparisons combined using AND, OR and NOT.
- DISTINCT** is an optional keyword indicating that the answer should not contain duplicates. Default is that duplicates are not eliminated! 删去重复元组

Principles of Database Systems, Xu Lizhen

9

Conceptual Evaluation Strategy

- Semantics of an SQL query defined in terms of the following conceptual evaluation strategy:
 - Compute the **cross-product** of *relation-list*. 笛卡尔积
 - Discard resulting tuples if they fail *qualifications*.
 - Delete attributes that are not in *target-list*.
 - If **DISTINCT** is specified, eliminate duplicate rows.
- This strategy is probably the **least efficient** way to compute a query! An optimizer will find more efficient strategies to compute *the same answers*.

Principles of Database Systems, Xu Lizhen

10

Simple Example

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid AND R.bid=103
```

(sid)	sname	rating	age	(sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

result

Principles of Database Systems, Xu Lizhen

11

A Note on Range Variables

- Really needed only if the same relation appears twice in the FROM clause. The previous query can also be written as:

```
SELECT S.sname
FROM Sailors S, Reserves R
WHERE S.sid=R.sid AND bid=103
```

OR

```
SELECT sname
FROM Sailors, Reserves
WHERE Sailors.sid=Reserves.sid
AND bid=103
```

It is good style, however, to use range variables always!

Principles of Database Systems, Xu Lizhen

12



Find sailors who've reserved at least one boat

```
SELECT S.sid
FROM Sailors S, Reserves R
WHERE S.sid=R.sid
```

- Would adding DISTINCT to this query make a difference?
- What is the effect of replacing S.sid by S.sname in the SELECT clause? Would adding DISTINCT to this variant of the query make a difference?



Expressions and Strings

```
SELECT S.age, age1=S.age-5, 2*S.age AS age2
FROM Sailors S
WHERE S.sname LIKE 'B_%B'
```

- Illustrates use of arithmetic expressions and string pattern matching: Find triples (of ages of sailors and two fields defined by expressions) for sailors whose names begin and end with B and contain at least three characters.
- AS and  are two ways to name fields in result.
- LIKE is used for string matching. '_' stands for any one character and '%' stands for 0 or more arbitrary characters.



Find sid's of sailors who've reserved a red or a green boat

- UNION**: Can be used to compute the union of any two *union-compatible* sets of tuples (which are themselves the result of SQL queries).
- If we replace OR by AND in the first version, what do we get?
- Also available: EXCEPT (What do we get if we replace UNION by EXCEPT?)

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
AND (B.color='red' OR B.color='green')
```

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
AND B.color='red'
```

```
UNION
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
AND B.color='green'
```



Find sid's of sailors who've reserved a red and a green boat

- INTERSECT**: Can be used to compute the intersection of any two *union-compatible* sets of tuples.
- Included in the SQL/92 standard, but some systems don't support it.
- Contrast symmetry of the UNION and INTERSECT queries with how much the other versions differ.

```
SELECT S.sid
FROM Sailors S, Boats B1, Reserves R1,
Boats B2, Reserves R2
WHERE S.sid=R1.sid AND R1.bid=B1.bid
AND S.sid=R2.sid AND R2.bid=B2.bid
AND (B1.color='red' AND B2.color='green')
```

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
AND B.color='red'
```

```
INTERSECT
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid
AND B.color='green'
```



Nested Queries 嵌套查询

Find names of sailors who've reserved boat #103:

```
SELECT S.sname
FROM Sailors S
WHERE S.sid IN (SELECT R.sid
FROM Reserves R
WHERE R.bid=103)
```

- A very powerful feature of SQL: a WHERE clause can itself contain an SQL query! (Actually, so can FROM and HAVING clauses.)
- To find sailors who've *not* reserved #103, use NOT IN.
- To understand semantics of nested queries, think of a *nested loops* evaluation: For each Sailors tuple, check the qualification by computing the subquery.



Nested Queries with Correlation

Find names of sailors who've reserved boat #103:

```
SELECT S.sname
FROM Sailors S
WHERE EXISTS (SELECT *
FROM Reserves R
WHERE R.bid=103 AND S.sid=R.sid)
```

两层循环

- EXISTS is another set comparison operator, like IN.
- Illustrates why, in general, subquery must be re-computed for each Sailors tuple.
- How to find names of sailors who've reserved boat #103 and reserved only one time?

Nested Queries with Correlation

- Find IDs of boats **which are reserved by only one sailor.**

```
SELECT bid
FROM Reserves R1
WHERE bid NOT IN (
    SELECT bid
    FROM Reserves R2
    WHERE R2.sid = R1.sid)
```

经典利用自联结 判断唯一性

More on Set-Comparison Operators

- We've already seen IN, EXISTS and UNIQUE. Can also use **NOT IN**, **NOT EXISTS** and **NOT UNIQUE**.
- Also available: **op ANY, op ALL, op IN**
 $<, >, =, \leq, \geq, \neq$
- Find sailors whose rating is greater than that of some sailor called Horatio:

```
SELECT *
FROM Sailors S
WHERE S.rating > ANY (SELECT S2.rating
    FROM Sailors S2
    WHERE S2.sname='Horatio')
```

Rewriting INTERSECT Queries Using IN

Find sid's of sailors who've reserved both a red and a green boat:

```
SELECT S.sid
FROM Sailors S, Boats B, Reserves R
WHERE S.sid=R.sid AND R.bid=B.bid AND B.color='red'
AND S.sid IN (SELECT S2.sid
    FROM Sailors S2, Boats B2, Reserves R2
    WHERE S2.sid=R2.sid AND R2.bid=B2.bid
    AND B2.color='green')
```

- Similarly, EXCEPT queries re-written using NOT IN.
- To find *names* (not *sid*'s) of Sailors who've reserved both red and green boats, just replace *S.sid* by *S.sname* in SELECT clause. (What about INTERSECT query?)

Division in SQL

Find sailors **who've reserved all boats.**

Solution 1:

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS
    ((SELECT B.bid
    FROM Boats B)
    EXCEPT
    (SELECT R.bid
    FROM Reserves R
    WHERE R.sid=S.sid))
```

不存在没有订过的船

Division in SQL

Solution 2:

Let's do it the hard way, **without EXCEPT:**

```
SELECT S.sname
FROM Sailors S
WHERE NOT EXISTS (SELECT B.bid
    FROM Boats B
    WHERE NOT EXISTS (SELECT R.bid
    FROM Reserves R
    WHERE R.bid=B.bid
    AND R.sid=S.sid))
```

Sailors S such that ...
 there is no boat B without ...
 a Reserves tuple showing S reserved B

Aggregate Operators

- Significant extension of relational algebra.
 - COUNT (*)
 - COUNT ([DISTINCT] A) 计算属性A的多少个[不同的]值
 - SUM ([DISTINCT] A)
 - AVG ([DISTINCT] A)
 - MAX (A)
 - MIN (A)
- A is single column

Examples of Aggregate Operators

```
SELECT COUNT (*)      SELECT COUNT (DISTINCT S.rating)
FROM Sailors S        FROM Sailors S
                      WHERE S.sname='Bob'
```

```
SELECT AVG (S.age)    SELECT AVG (DISTINCT S.age)
FROM Sailors S        FROM Sailors S
WHERE S.rating=10     WHERE S.rating=10
```

```
SELECT S.sname
FROM Sailors S
WHERE S.rating= (SELECT MAX(S2.rating)
                 FROM Sailors S2)
```

Principles of Database Systems, Xu Lizhen

25

Find name and age of the oldest sailor(s)

- The first query is illegal! (We'll look into the reason a bit later, when we discuss **GROUP BY**.)
- The third query is equivalent to the second query, and is allowed in the SQL/92 standard, but is not supported in some systems.

```
SELECT S.sname, MAX (S.age)
FROM Sailors S

SELECT S.sname, S.age
FROM Sailors S
WHERE S.age =
      (SELECT MAX (S2.age)
       FROM Sailors S2)

SELECT S.sname, S.age
FROM Sailors S
WHERE (SELECT MAX (S2.age)
      FROM Sailors S2)
      = S.age
```

Principles of Database Systems, Xu Lizhen

26

Motivation for Grouping

- So far, we've applied aggregate operators to all (qualifying) tuples. Sometimes, we want to apply them to each of several *groups* of tuples.
- Consider: *Find the age of the youngest sailor for each rating level.*
 - In general, we don't know how many rating levels exist, and what the rating values for these levels are!
 - Suppose we know that rating values go from 1 to 10; we can write 10 queries that look like this (!):

```
For i = 1, 2, ..., 10:      SELECT MIN (S.age)
                           FROM Sailors S
                           WHERE S.rating = i
```

Principles of Database Systems, Xu Lizhen

27

Queries With GROUP BY and HAVING

```
SELECT [DISTINCT] target-list
FROM relation-list
WHERE qualification
GROUP BY grouping-list
HAVING group-qualification
```

- The *target-list* contains
 - (i) **attribute** names
 - (ii) terms with **aggregate** operations (e.g., MIN (S.age)).
- The attribute list (i) must be a subset of *grouping-list*. Intuitively, each answer tuple corresponds to a *group*, and these attributes must have a single value per group. (A *group* is a set of tuples that have the same value for all attributes in *grouping-list*.)

Principles of Database Systems, Xu Lizhen

28

Conceptual Evaluation

- The cross-product of *relation-list* is computed, tuples that fail *qualification* are discarded, 'unnecessary' fields are deleted, and the remaining tuples are partitioned into groups by the value of attributes in *grouping-list*.
- The *group-qualification* is then applied to eliminate some groups. Expressions in *group-qualification* must have a **single value per group**!
 - In fact, an attribute in *group-qualification* that is not an argument of an aggregate op also appears in *grouping-list*. (SQL does not exploit primary key semantics here!)
- One answer tuple is generated per qualifying group.

Principles of Database Systems, Xu Lizhen

29

Find age of the youngest sailor with age ≥ 18, for each rating with at least 2 such sailors

```
SELECT S.rating, MIN (S.age) AS minage
FROM Sailors S
WHERE S.age >= 18
GROUP BY S.rating
HAVING COUNT (*) > 1
```

Answer relation:

rating	minage
3	25.5
7	35.0
8	25.5

Sailors instance:

sid	sname	rating	age
22	dustin	7	45.0
29	brutus	1	33.0
31	lubber	8	55.5
32	andy	8	25.5
58	rusty	10	35.0
64	horatio	7	35.0
71	zorba	10	16.0
74	horatio	9	35.0
85	art	3	25.5
95	bob	3	63.5
96	frodo	3	25.5

Principles of Database Systems, Xu Lizhen

30

- ★ Find age of the youngest sailor with age ≥ 18 ,
✖ for each rating with at least 2 such sailors.

rating	age	rating	age	rating	minage
7	45.0	1	33.0	3	25.5
1	33.0	3	25.5	3	63.5
8	55.5	3	25.5	7	45.0
8	25.5	7	35.0	7	35.0
10	35.0	8	55.5	8	25.5
7	35.0	8	25.5	9	35.0
10	16.0	9	35.0	10	35.0
9	35.0				
3	25.5				
3	63.5				
3	25.5				

Principles of Database Systems, Xu Lizhen

31

- ★ Find age of the youngest sailor with age ≥ 18 , for each rating with
✖ at least 2 such sailors and with every sailor under 60.

HAVING COUNT (*) > 1 AND EVERY (S.age <=60)

rating	age	rating	age	rating	minage
7	45.0	1	33.0	7	35.0
1	33.0	3	25.5	8	25.5
8	55.5	3	63.5		
8	25.5	3	25.5		
10	35.0	7	45.0		
7	35.0	7	35.0		
10	16.0	8	55.5		
9	35.0	8	25.5		
3	25.5	9	35.0		
3	63.5	10	35.0		
3	25.5				

What is the result of changing EVERY to ANY?

Principles of Database Systems, Xu Lizhen

32

- ★ For each red boat, find the number of
✖ reservations for this boat

```
SELECT B.bid, COUNT (*) AS scount
FROM Boats B, Reserves R
WHERE R.bid=B.bid AND B.color='red'
GROUP BY B.bid
```

- Grouping over a join of two relations.
- What do we get if we remove *B.color='red'* from the WHERE clause and add a HAVING clause with this condition?

Principles of Database Systems, Xu Lizhen

33

- ★ Find age of the youngest sailor with age > 18,
✖ for each rating with at least 2 sailors (of any age)

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
WHERE S.age > 18
GROUP BY S.rating
HAVING 1 < (SELECT COUNT (*)
FROM Sailors S2
WHERE S2.rating = S.rating)
```

rating	minage
3	25.5
7	35.0
8	25.5
10	35.5

- Shows HAVING clause can also contain a sub-query.
- Compare this with the query where we considered only ratings with 2 sailors over 18!
- What if HAVING clause is replaced by:
 - HAVING COUNT(*) > 1

Principles of Database Systems, Xu Lizhen

34

- ★ Find those ratings for which the average age is
✖ the minimum over all ratings

- Aggregate operations cannot be nested! WRONG:

```
SELECT S.rating
FROM Sailors S
WHERE S.age = (SELECT MIN (AVG (S2.age))
FROM Sailors S2)
```

- Correct solution (in SQL/92):

```
SELECT Temp.rating
FROM (SELECT S.rating, AVG (S.age) AS avgage
FROM Sailors S
GROUP BY S.rating) AS Temp
WHERE Temp.avgage = (SELECT MIN (Temp.avgage)
FROM Temp)
```

Principles of Database Systems, Xu Lizhen

35

- ★ Null Values

- Field values in a tuple are sometimes *unknown* (e.g., a rating has not been assigned) or *inapplicable* (e.g., no spouse's name).

SQL provides a special value *null* for such situations.

- The presence of *null* complicates many issues. E.g.:

Special operators needed to check if value is/is not *null*.

仅取True Is *rating* > 8 true or false when *rating* is equal to *null*? What about AND, OR and NOT connectives?

We need a 3-valued logic (true, false and *unknown*).

Meaning of constructs must be defined carefully. (e.g., WHERE clause eliminates rows that don't evaluate to true.)

New operators (in particular, *outer joins*) possible/needed.

Principles of Database Systems, Xu Lizhen

36

NOT

x	NOT x
true	false
unknown	unknown
false	true

AND

AND	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

OR

OR	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false



Some New Features of SQL

- **CAST expression**
- CASE expression
- Sub-query
- Outer Join
- Recursion



CAST Expression 强制类型转换

→ CAST → ((Expression) AS (Data type)) →
 NULL

- Change the expression to the target data type
- Valid target type
- Use
 - Match function parameters
 substr(string1, CAST(x AS Integer), CAST(y AS Integer))
 - **Change precision** while calculating
 CAST (elevation AS Decimal (5,0))
 - Assign a data type to NULL value



CAST Expression

- Example:
 Students (name, school)
 Soldiers (name, service)

 CREATE VIEW prospects (name, school, service) AS
 SELECT name, school, CAST(NULL AS Varchar(20))
 FROM Students
 UNION
 SELECT name, CAST(NULL AS Varchar(20)), service
 FROM Soldiers ; **强行合并**



Some New Features of SQL

- CAST expression
- **CASE expression**
- Sub-query
- Outer Join
- Recursion



CASE Expression

- Simple form :
 Officers (name, status, rank, title)
 SELECT name, CASE status
 WHEN 1 THEN 'Active Duty'
 WHEN 2 THEN 'Reserve'
 WHEN 3 THEN 'Special Assignment'
 WHEN 4 THEN 'Retired'
 ELSE 'Unknown'
 END AS status
 FROM Officers ;



CASE Expression

- General form (use searching condition):
 Machines (serialno, type, year, hours_used, accidents)
- Find the rate of the accidents of "chain saw" in the whole accidents :
 SELECT sum CASE
 WHEN type='chain saw' THEN accidents
 ELSE 0e0
 END / sum (accidents)
 FROM Machines;



CASE Expression

- Find the average accident rate of every kind of equipment :

```
SELECT type, CASE
    WHEN sum(hours_used)>0 THEN
        sum(accidents)/sum(hours_used)
    ELSE NULL
END AS accident_rate
FROM Machines
GROUP BY type;
```

(Because some equipments maybe not in use at all, their hours_used is 0. Use CASE can prevent the expression divided by 0.)



CASE Expression

- Compared with

```
SELECT type, sum(accidents)/sum(hours_used)
FROM Machines
GROUP BY type
HAVING sum(hours_used)>0;
```



Some New Features of SQL

- CAST expression
- CASE expression
- Sub-query**
- Outer Join
- Recursion



Sub-query

- Embedded query & embedded query with correlation
- The functions of sub-queries have been enhanced in new SQL standard. Now they can be used in SELECT and FROM clause
 - Scalar sub-query 一个数值
 - Table expression 表表达式
 - Common table expression



Scalar Sub-query

- The **result of a sub-query is a single value**. It can be used in the place where a value can occur.

- Find the departments whose average bonus is higher than average salary :

```
SELECT d.deptname, d.location
FROM dept AS d
WHERE (SELECT avg(bonus)
    FROM emp
    WHERE deptno=d.deptno)
> (SELECT avg(salary)
    FROM emp
    WHERE deptno=d.deptno)
```



Scalar Sub-query

- List the deptno, deptname, and the max salary of all departments located in New York :

```
SELECT d.deptno, d.deptname, (SELECT MAX (salary)
    FROM emp
    WHERE deptno=d.deptno) AS maxpay
FROM dept AS d
WHERE d.location = 'New York' ;
```



Table Expression

- The result of a sub-query is a table. It can be used in the place where a table can occur.

```
SELECT startyear, avg(pay)
FROM (SELECT name, salary+bonus AS pay,
      year(startdate) AS startyear
      FROM emp) AS emp2
GROUP BY startyear;
```

- Find departments whose total payment is greater than 200000

```
SELECT deptno, totalpay
FROM (SELECT deptno, sum(salary)+sum(bonus) AS totalpay
      FROM emp
      GROUP BY deptno) AS payroll
WHERE totalpay>200000;
```

- Table expressions are temporary views in fact.



Common Table Expression

- In some complex query, a table expression may need occurring more than one time in the same SQL statements. Although it is permitted, the efficiency is low and there maybe inconsistency problem.

- WITH** clause can be used to define a common table expression. In fact, it defines a temporary view.

- Find the department who has the highest total payment :



Common Table Expression

- Find the department who has the highest total payment : 公共表表达式 = 临时视图

```
WITH payroll (deptno, totalpay) AS
(SELECT deptno, sum(salary)+sum(bonus)
 FROM emp
 GROUP BY deptno)
```

```
SELECT deptno
FROM payroll
WHERE totalpay = (SELECT max(totalpay)
                  FROM payroll);
```

- Common table expression mainly used in queries which need multi level focuses.



Common Table Expression

- Find department pairs, in which the first department's average salary is more than two times of the second one's :

```
WITH deptavg (deptno, avgсал) AS
(SELECT deptno, avg(salary)
 FROM emp
 GROUP BY deptno)
SELECT d1.deptno, d1.avgсал, d2.deptno, d2.avgсал
FROM deptavg AS d1, deptavg AS d2
WHERE d1.avgсал>2*d2.avgсал;
```



Some New Features of SQL

- CAST expression
- CASE expression
- Sub-query
- Outer Join**
- Recursion



外连接

Outer Join

Teacher (name, rank)
Course (subject, enrollment, quarter, teacher)

```
WITH
innerjoin(name, rank, subject, enrollment) AS
(SELECT t.name, t.rank, c.subject, c.enrollment
 FROM teachers AS t, courses AS c
 WHERE t.name=c.teacher AND c.quarter='Fall 96'),
teacher-only(name, rank) AS
(SELECT name, rank
 FROM teachers
 EXCEPT ALL
 SELECT name, rank
 FROM innerjoin),
course-only(subject, enrollment) AS
(SELECT subject, enrollment
 FROM courses
 EXCEPT ALL
 SELECT subject, enrollment
 FROM innerjoin)
```



Outer Join

```
SELECT name, rank, subject, enrollment
FROM innerjoin
UNION ALL
SELECT name, rank,
       CAST (NULL AS Varchar(20)) AS subject,
       CAST (NULL AS Integer) AS enrollment
FROM teacher-only
UNION ALL
SELECT CAST (NULL AS Varchar(20)) AS name,
       CAST (NULL AS Varchar(20)) AS rank,
       subject, enrollment
FROM course-only ;
```



Some New Features of SQL

- CAST expression
- CASE expression
- Sub-query
- Outer Join
- **Recursion**



Recursion

- If a common table expression uses itself in its definition, this is called recursion. It can calculate a complex recursive inference in one SQL statement.
- Find all employees under the management of Hoover and whose salary is more than 100000

直接&间接

```
WITH agents (name, salary) AS
  ((SELECT name, salary
    FROM FedEmp
    WHERE manager='Hoover')
   UNION ALL
   (SELECT f.name, f.salary
    FROM agents AS a, FedEmp AS f
    WHERE f.manager = a.name))
SELECT name
FROM agents
WHERE salary > 100000 ;
```

在定义种包含了自己

递归

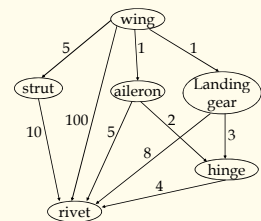


Recursive Calculation

- A classical "parts searching problem"

Components

Part	Subpart	QTY
wing	strut	5
wing	aileron	1
wing	landing gear	1
wing	rivet	100
strut	rivet	10
aileron	hinge	2
aileron	rivet	5
landing gear	hinge	3
landing gear	rivet	8
hinge	rivet	4



Directed acyclic graph, which assures the recursion can be stopped



Recursive Calculation

- Find how much rivets are used in one wing?
- A temporary view is defined to show the list of each subpart's quantity used in a specified part :

```
WITH wingpart (subpart, qty) AS
  ((SELECT subpart, qty
    FROM components
    WHERE part='wing')
   UNION ALL
   (SELECT c.subpart, w.qty*c.qty
    FROM wingpart w, components c
    WHERE w.subpart=c.part))
```

wingpart		
Subpart	QTY	
strut	5	Used directly
aileron	1	Used directly
landing gear	1	Used directly
rivet	100	Used directly
rivet	50	Used on strut
hinge	2	Used on aileron
rivet	5	Used on aileron
hinge	3	Used on landing gear
rivet	8	Used on landing gear
rivet	8	Used on aileron hinges
rivet	12	on L.G hinges



Recursive Calculation

- Find how much rivets are used in one wing?

```
WITH wingpart (subpart, qty) AS
  ((SELECT subpart, qty
    FROM components
    WHERE part='wing')
   UNION ALL
   (SELECT c.subpart, w.qty*c.qty
    FROM wingpart w, components c
    WHERE w.subpart=c.part))
SELECT sum(qty) AS qty
FROM wingpart
WHERE subpart='rivet' ;
```

- The result is :

qty
183



Recursive Calculation

- Find all subparts and their total quantity needed to assemble a wing :

```
WITH wingpart (subpart, qty) AS
((SELECT subpart, qty
FROM components
WHERE part='wing')
UNION ALL
(SELECT c.subpart, w.qty*c.qty ---recursive qry
FROM wingpart w, components c
WHERE w.subpart=c.part))
SELECT subpart, sum(qty) AS qty
FROM wingpart
Group BY subpart ;
```

- The result is :

subpart	qty
strut	5
aileron	1
landing gear	1
hinge	5
rivet	183

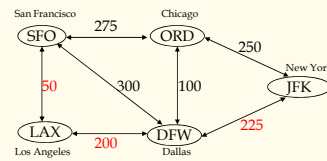
Principles of Database Systems, Xu Lizhen

61



Recursive Search

- Typical airline route searching problem
- Find the lowest total cost route from SFO to JFK



Flights			
FlightNo	Origin	Destination	Cost
HY 120	DFW	JFK	225
HY 130	DFW	LAX	200
HY 140	DFW	ORD	100
HY 150	DFW	SFO	300
HY 210	JFK	DFW	225
HY 240	JFK	ORD	250
HY 310	LAX	DFW	200
HY 350	LAX	SFO	50
HY 410	ORD	DFW	100
HY 420	ORD	JFK	250
HY 450	ORD	SFO	275
HY 510	SFO	DFW	300
HY 530	SFO	LAX	50
HY 540	SFO	ORD	275

Principles of Database Systems, Xu Lizhen

62



Recursive Search

```
WITH trips (destination, route, nsegs, totalcost) AS
((SELECT destination, CAST(destination AS varchar(20)), 1, cost
FROM flights
WHERE origin='SFO')
UNION ALL
(SELECT f.destination,
CAST(t.route || ',' || f.destination AS varchar(20)),
t.nsegs+1, t.totalcost+f.cost
FROM trips t, flights f
WHERE t.destination=f.origin
AND f.destination<>'SFO'
AND f.origin<>'JFK'
AND t.nsegs<=3))
SELECT route, totalcost
FROM trips
WHERE destination='JFK' AND totalcost=
(SELECT min(totalcost)
FROM trips
WHERE destination='JFK') ;
```

Principles of Database Systems, Xu Lizhen

63



Result

Trips			
Destination	Route	Nsegs	Totalcost
DFW	DFW	1	300
ORD	ORD	1	275
LAX	LAX	1	50
JFK	DFW, JFK	2	525
LAX	DFW, LAX	2	500
ORD	DFW, ORD	2	400
DFW	LAX, DFW	2	250
DFW	ORD, DFW	2	375
JFK	ORD, JFK	2	525
DFW	DFW, LAX, DFW	3	700
DFW	DFW, ORD, DFW	3	500
JFK	DFW, ORD, JFK	3	650
LAX	LAX, DFW, LAX	3	450
JFK	LAX, DFW, JFK	3	475
ORD	LAX, DFW, ORD	3	350
LAX	ORD, DFW, LAX	3	575
JFK	ORD, DFW, JFK	3	600
ORD	ORD, DFW, ORD	3	475

Final result

route	totalcost
LAX, DFW, JFK	475

Principles of Database Systems, Xu Lizhen

64



Recursive Search

- Only change the final query slightly, the least transfer time routes can be found :

```
... ..
SELECT route, totalcost
FROM trips
WHERE destination='JFK' AND nsegs=
(SELECT min(nsegs)
FROM trips
WHERE destination='JFK') ;
```

Final result

route	totalcost
DFW, JFK	525
ORD, JFK	525

Principles of Database Systems, Xu Lizhen

65



Data Manipulation Language

- Insert
 - Insert a tuple into a table
 - INSERT INTO EMPLOYEES VALUES ('Smith', 'John', '1980-06-10', 'Los Angles', 16, 45000);
- Delete
 - Delete tuples fulfill qualifications
 - DELETE FROM Person WHERE LastName = 'Rasmussen' ;
- Update
 - Update the attributes' value of tuples fulfill qualifications
 - UPDATE Person SET Address = 'Zhongshan 23', City = 'Nanjing' WHERE LastName = 'Wilson' ;

Principles of Database Systems, Xu Lizhen

66



View in SQL

- General view
 - Virtual tables derived from base tables
 - Logical data independence
 - Security of data
 - Update problems of view
- Temporary view and recursive query
 - WITH
 - RECURSIVE

Principles of Database Systems, Xu Lizhen



Update problems of view

- CREATE VIEW YoungSailor AS
SELECT sid, sname, rating
FROM Sailors
WHERE age<26;
- CREATE VIEW Ratingavg AS
SELECT rating, AVG(age)
FROM Sailors
GROUP BY rating;

Principles of Database Systems, Xu Lizhen

68

可更新视图有以下三条规则：

(1) 若视图是基于多个表使用联接操作而导出的，那么对这个视图执行更新操作时，每次只能影响其中的一个表。

(2) 若视图导出时包含有分组和聚合操作，则不允许对这个视图执行更新操作。

(3) 若视图是从一个表经选择、投影而导出的，并在视图包含了表的主键字或某个候选键，这类视图称为‘行列子集视图’。对这类视图可执行更新操作。



Embedded SQL

- In order to access database in programs, and take further process to the query results, need to combine SQL and programming language (such as C / C++, etc.)
- Problems should be solved:
 - How to accept SQL statements in programming language
 - How to exchange data and messages between programming language and DBMS
 - The query result of DBMS is a set, how to transfer it to the variables in programming language
 - The data type of DBMS and programming language may not the same exactly.

Principles of Database Systems, Xu Lizhen

69



General Solutions

- Embedded SQL
 - The most basic method. Through pre-compiling, transfer the embedded SQL statements to inner library functions call to access database.
- Programming APIs
 - Offer a set of library functions or DLLs to programmer directly, linking with application program while compiling.
- Class Library
 - Supported after emerging of OOP. Envelope the library functions to access database as a set of class, offering easier way to treat database in programming language.

Principles of Database Systems, Xu Lizhen

70

程序设计语言和数据库系统之间存在所谓的“阻抗不匹配”问题，关于此问题下说法正确的是：

它和程序设计语言与DBMS之间的数据交换有关

数据库的处理对象是集合，而程序设计语言是变量

一个DBMS所支持的数据类型与程序设计语言支持的不完全一致



Usage of Embedded SQL (in C)

- SQL statements can be used in C program directly:
 - **Begin** with *EXEC SQL*, end with ‘;’
 - Though *host variables* to transfer information between C and SQL. Host variables should be defined begin with *EXEC SQL*.
 - In SQL statements, should add ‘:’ before host variables to distinguish with SQL’s own variable or attributes’ name.
 - In host language (such as C), host variables are used as general variables.
 - Can’t define host variables as Array or Structure.
 - A special host variable, SQLCA (SQL Communication Area)
EXEC SQL INCLUDE SQLCA
 - Use *SQLCA.SQLCODE* to justify the *state* of result.
 - Use *indicator* (short int) to treat *NULL* in host language.

Principles of Database Systems, Xu Lizhen

71



Example of *host variables defining*

```
EXEC SQL BEGIN DECLARE SECTION;
char SNO[7];
char GIVENSNO[7];
char CNO[6];
char GIVENCNO[6];
float GRADE;
short GRADEI; /*indicator of GRADE*/
EXEC SQL END DECLARE SECTION;
```

Principles of Database Systems, Xu Lizhen

72

Executable Statements

- CONNECT
 - EXEC SQL CONNECT :uid IDENTIFIED BY :pwd;
- Execute DDL or DML Statements
 - EXEC SQL INSERT INTO SC(SNO,CNO,GRADE) VALUES(:SNO, :CNO, :GRADE);
- Execute Query Statements
 - EXEC SQL SELECT GRADE INTO :GRADE :GRADEI 单元组查询 FROM SC WHERE SNO=:GIVENSNO AND CNO=:GIVENCNO;
- Because {SNO,CNO} is the key of SC, the result of this query has only one tuple. How to treat result if it has a set of tuples?

Cursor 查询结果为多元组

1. Define a cursor
 - EXEC SQL DECLARE <cursor name> CURSOR FOR SELECT ... FROM ... WHERE ... 先为一个查询定义Cursor
2. EXEC SQL OPEN <cursor name>
 - Some like open a file
3. Fetch data from cursor
 - EXEC SQL FETCH <cursor name> INTO :hostvar1, :hostvar2, ...;
4. SQLCA.SQLCODE will return 100 when arriving the end of cursor
5. CLOSE CURSOR <cursor name>

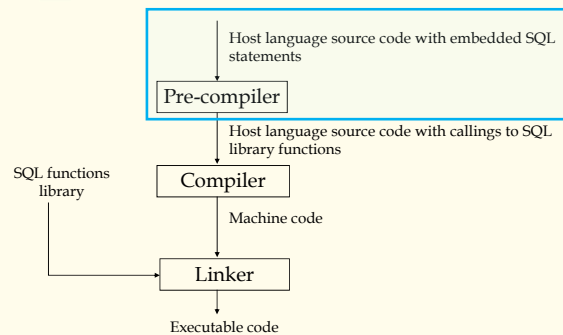
Example of Query with Cursor

```

:
EXEC SQL DECLARE C1 CURSOR FOR
  SELECT SNO, GRADE
  FROM SC
  WHERE CNO = :GIVENCNO;
EXEC SQL OPEN C1;
if (SQLCA.SQLCODE<0) exit(1); /* There is error in query*/
while (1) {
  EXEC SQL FETCH C1 INTO :SNO, :GRADE :GRADEI
  if (SQLCA.SQLCODE==100) break;
  /* treat data fetched from cursor, omitted*/
  :
}
EXEC SQL CLOSE C1;
:

```

Conceptual Evaluation



Dynamic SQL

- In above embedded SQL, the SQL statements must be written before compiling. But in some applications, the SQL statement can't be decided in ahead, they need to be built dynamically while the program running.
- Dynamic SQL is supported in SQL standard and most RDBMS products
 - Dynamic SQL executed directly
 - Dynamic SQL with dynamic parameters
 - Dynamic SQL for query

Dynamic SQL executed directly

- Only used in the execution of non query SQL statements
 - EXEC SQL BEGIN DECLARE SECTION;
 - char sqlstring[200];
 - EXEC SQL END DECLARE SECTION;
 - char cond[150];
 - strcpy(sqlstring, "DELETE FROM STUDENT WHERE ");
 - printf(" Enter search condition :");
 - scanf("%s", cond);
 - strcat(sqlstring, cond);
 - EXEC SQL EXECUTE IMMEDIATE :sqlstring;



Dynamic SQL with dynamic parameters

- Only used in the execution of non query SQL statements. Use **place holder** to realize dynamic parameter in SQL statement. Some like the macro processing method in C.

```

:
EXEC SQL BEGIN DECLARE SECTION;
char sqlstring[200];
int birth_year;
EXEC SQL END DECLARE SECTION;
strcpy(sqlstring, "DELETE FROM STUDENT WHERE
YEAR(BDATE) <= :y;");
printf(" Enter birth year for delete :");
scanf("%d", &birth_year);
EXEC SQL PREPARE PURGE FROM :sqlstring;
EXEC SQL EXECUTE PURGE USING :birth_year;
:

```

就一个
名字



Dynamic SQL for query

- Used to form query statement dynamically

```

:
EXEC SQL BEGIN DECLARE SECTION;
char sqlstring[200];
char SNO[7];
float GRADE;
short GRADEI;
char GIVENCNO[6];
EXEC SQL END DECLARE SECTION;
char orderby[150];
strcpy(sqlstring, "SELECT SNO, GRADE FROM SC WHERE CNO= :c");
printf(" Enter the ORDER BY clause :");
scanf("%s", orderby);
strcat(sqlstring, orderby);
printf(" Enter the course number :");
scanf("%s", GIVENCNO);
EXEC SQL PREPARE query FROM :sqlstring;
EXEC SQL DECLARE grade_cursor CURSOR FOR query;
EXEC SQL OPEN grade_cursor USING :GIVENCNO;

```



Dynamic SQL for query (Cont.)

```

if (SQLCA.SQLCODE < 0) exit(1); /* There is error in query*/
while (1) {
    EXEC SQL FETCH grade_cursor INTO :SNO, :GRADE :GRADEI
    if (SQLCA.SQLCODE == 100) break;
    /* treat data fetched from cursor, omitted*/
    :
}
EXEC SQL CLOSE grade_cursor;
:

```



Stored Procedure

- Used to improve performance and facilitate users. With it, user can take frequently used database access program as a procedure, and store it in the database after compiling, then call it directly while need.
 - Make user convenient. User can call them directly and don't need code again. They are reusable.
 - Improve performance. The stored procedures have been compiled, so they don't need parsing and query optimization again while being used.
 - Expand function of DBMS. (can write script)



Example of a Stored Procedure

```

EXEC SQL
CREATE PROCEDURE drop_student
(IN student_no CHAR(7),
OUT message CHAR(30))
BEGIN ATOMIC
DELETE FROM STUDENT
WHERE SNO=student_no;
DELETE FROM SC
WHERE SNO=student_no;
SET message=student_no || 'dropped';
END;
EXEC SQL

:
CALL drop_student(...); /* call this stored procedure later*/
:

```


4. Database Management Systems*

Contents

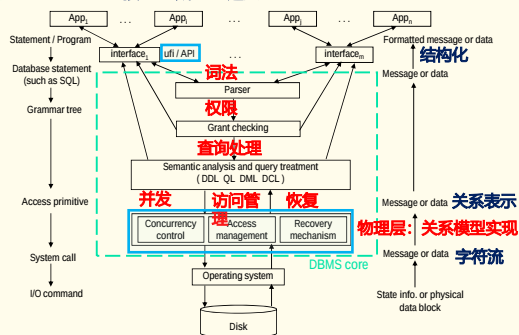
- The Architecture of DBMS
 - The components of DBMS core
 - The process structure of DBMS
- Database Access Management
- Query Optimization
- Transaction Management 事务管理
 - Recovery
 - Concurrent Control

Principles of Database Systems, Xu Lizhen

2

4.1 The Components of DBMS Core

接口生成SQL送入DBMS core



Principles of Database Systems, Xu Lizhen

3

4.2 The Process Structure of DBMS

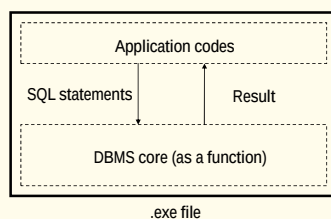
- Single process structure
- Multi processes structure
- Multi threads structure
- Communication protocols between processes / threads

Principles of Database Systems, Xu Lizhen

4

Single process structure

- The application program is compiled with DBMS core as a single .exe file, running as a single process.

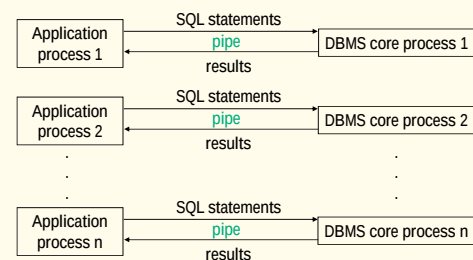


Principles of Database Systems, Xu Lizhen

5

Multi processes structure

- One application process corresponding to one DBMS core process

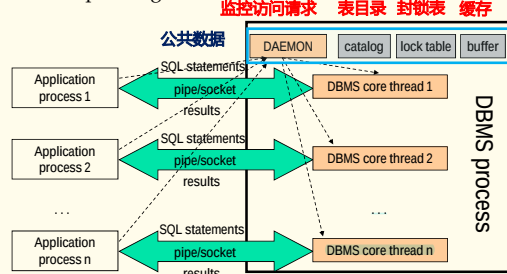


Principles of Database Systems, Xu Lizhen

6

Multi threads structure

- Only one DBMS process, every application process corresponding to a DBMS core thread.



4.3 Database Access Management

The access to database is transferred to the operations on files (of OS) eventually. The file structure and access route offered on it will affect the speed of data access directly. It is impossible that one kind of file structure will be effective for all kinds of data access.

- Access types 访问类型
- File organization 文件结构
- Index technique 索引技术
- Access primitives

Access Types

- Query all or most records of a file (>15%) 查询时涉及大部分元组
- Query some special record 特定查询
- Query some records (<15%) 设计一部分
- Scope query 特殊范围查询
- Update 写操作

File Organization

- Heap file: records stored according to their inserted order, and retrieved sequentially. This is the most basic and general form of file organization. 顺序扫描
- Direct file: the record address is mapped through hash function according to some attribute's value. Hash 文件
- Indexed file: index + heap file/cluster 常用
- Dynamic hashing: p115 动态调整hash域
- Grid structure file: p118 (suitable for multi attributes queries) 多维数组 逻辑顺序和物理顺序是不同, RD则可以
- Raw disk (notice the difference between the logical block and physical block of file. You can control physical blocks in OS by using raw disk)

Index Technique

- B+ Tree (✓✓) 最常用
- Clustering index (✓) 存储结果本身就可以作为索引
- Inverted file
- Dynamic hashing
- Grid structure file and partitioned hash function
- Bitmap index (used in data warehouse)
- Others

4.4 Query Optimization

"Rewrite" the query statements submitted by user first, and then decide the most effective operating method and steps to get the result. The goal is to gain the result of user's query with the lowest cost and in shortest time.

4.4.1 Summary of Query Optimization

- Algebra Optimization 代数优化 改变顺序等价变形
- Operation Optimization 操作优化



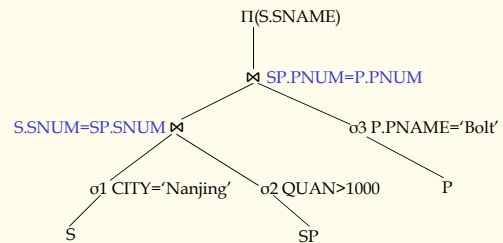
Example

S(SNUM, SNAME, CITY)
 SP(SNUM, PNUM, QUAN)
 P(PNUM, PNAME, WEIGHT, SIZE)

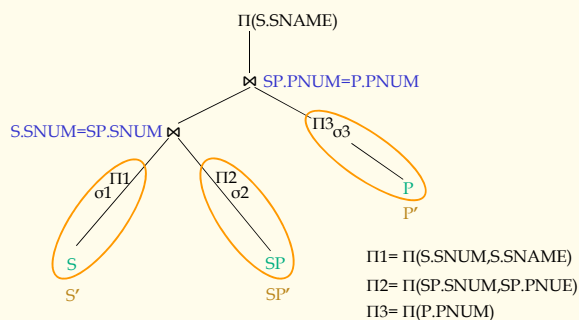
SELECT SNAME
 FROM S, SP, P
 WHERE S.SNUM=SP.SNUM AND
 SP.PNUM=P.PNUM AND
 S.CITY='Nanjing' AND
 P.PNAME='Bolt' AND
 SP.QUAN>1000;



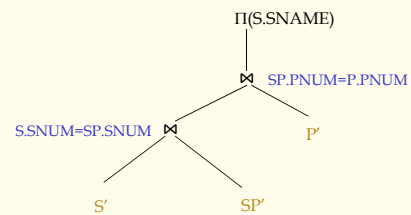
Query tree



After equivalent transform (Algebra optimization) :

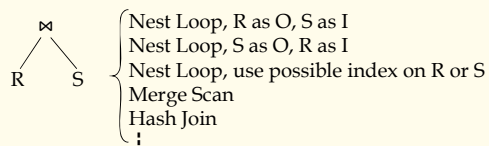


The result of equivalent transform



The operation optimization of the tree :

- Decide the order of two joins
- For every join operation, there are many computing method:



The goal of query optimization is to select a “good” solution from so many possible execution strategies. So it is a complex task.



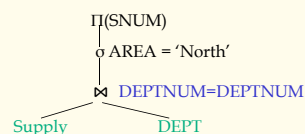
4.4.2 The Equivalent Transform of a Query

That is so called algebra optimization. It takes a series of transform on original query expression, and transform it into an equivalent, most effective form to be executed.

For example: $\Pi_{NAME, DEPT} \sigma_{DEPT=15}(EMP) \equiv \sigma_{DEPT=15} \Pi_{NAME, DEPT}(EMP)$

(1) Query tree

For example: $\Pi_{SNUM} \sigma_{AREA='NORTH'}(SUPPLY \bowtie_{DEPTNUM} DEPT)$



Leaves: relations
 Middle nodes: unary/binary operations
 Leaves → root: the executing order of operations

(2) The equivalent transform rules of relational algebra

- 1) Exchange rule of $\bowtie$: $E1 \bowtie E2 \equiv E2 \bowtie E1$
- 2) Combination rule of $\bowtie$: $E1 \bowtie (E2 \bowtie E3) \equiv (E1 \bowtie E2) \bowtie E3$
- 3) Cluster rule of Π : $\Pi_{A_1 \dots A_n}(\Pi_{B_1 \dots B_m}(E)) \equiv \Pi_{A_1 \dots A_n}(E)$, legal when $A_1 \dots A_n$ is the sub set of $\{B_1 \dots B_m\}$
- 4) Cluster rule of σ : $\sigma_{F_1}(\sigma_{F_2}(E)) \equiv \sigma_{F_1 \wedge F_2}(E)$
- 5) Exchange rule of σ and Π : $\sigma_F(\Pi_{A_1 \dots A_n}(E)) \equiv \Pi_{A_1 \dots A_n}(\sigma_F(E))$ if F includes attributes $B_1 \dots B_m$ which don't belong to $A_1 \dots A_n$, then $\Pi_{A_1 \dots A_n}(\sigma_F(E)) \equiv \Pi_{A_1 \dots A_n} \sigma_F(\Pi_{A_1 \dots A_n, B_1 \dots B_m}(E))$
- 6) If the attributes in F are all the attributes in $E1$, then $\sigma_F(E1 \bowtie E2) \equiv \sigma_F(E1) \bowtie E2$

if F in the form of $F1 \wedge F2$, and there are only $E1$'s attributes in $F1$, and there are only $E2$'s attributes in $F2$, then $\sigma_F(E1 \bowtie E2) \equiv \sigma_{F1}(E1) \bowtie \sigma_{F2}(E2)$

if F in the form of $F1 \wedge F2$, and there are only $E1$'s attributes in $F1$, while $F2$ includes the attributes both in $E1$ and $E2$, then $\sigma_F(E1 \bowtie E2) \equiv \sigma_{F2}(\sigma_{F1}(E1) \bowtie E2)$

- 7) $\sigma_F(E1 \cup E2) \equiv \sigma_F(E1) \cup \sigma_F(E2)$
- 8) $\sigma_F(E1 - E2) \equiv \sigma_F(E1) - \sigma_F(E2)$
- 9) Suppose $A_1 \dots A_n$ is a set of attributes, in which $B_1 \dots B_m$ are $E1$'s attributes, and $C_1 \dots C_k$ are $E2$'s attributes, then
 $\Pi_{A_1 \dots A_n}(E1 \bowtie E2) \equiv \Pi_{B_1 \dots B_m}(E1) \bowtie \Pi_{C_1 \dots C_k}(E2)$
- 10) $\Pi_{A_1 \dots A_n}(E1 \cup E2) \equiv \Pi_{A_1 \dots A_n}(E1) \cup \Pi_{A_1 \dots A_n}(E2)$

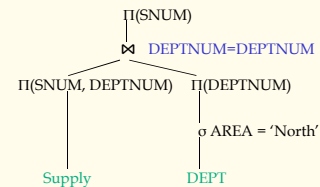
```
SELECT S.sname
FROM Sailors S
WHERE EXISTS (SELECT *
              FROM Reserves R
              WHERE R.bid=103 AND S.sid=R.sid)
```

? 解除嵌套
在代数优化阶段需要对应的优化规则

(3) Basic principles

The target of algebra optimization is to make the scale of the operands which involved in binary operations be as small as possible :

- ✓ Push down the unary operations as low as possible
- ✓ Look for and combine the common sub-expression



4.4.3 The Operation Optimization

How to find a "good" access strategy to compute the query improved by algebra optimization is introduced in this section:

- Optimization of select operation
- Optimization of project operation
- Optimization of set operation
- Optimization of join operation **三范式 最重要**
- Optimization of combined operations

Optimization of join operation

嵌套循环

- Nested loop: one relation acts as outer loop relation (O), the other acts as inner loop relation (I). For every tuple in O, scan I one time to check join condition. **内外循环关系**

Because the relation is accessed from disk in the unit of block, we can use block buffer to improve efficiency. For $R \bowtie S$, if let R as O, S as I, b_R is physical block number of R , b_S is physical block number of S , there are n_B block buffers in system ($n_B \geq 2$), and $n_B - 1$ buffers used for O, one buffer used for I, then the total disk access times needed to compute $R \bowtie S$ is:

$$b_R + \lceil b_R / (n_B - 1) \rceil \times b_S$$

Optimization of join operation

- **Merge scan**: order the relation R and S on disk in ahead, then we can compare their tuples in order, and both relation only need to scan one time. If R and S have not ordered in ahead, must consider the ordering cost to see if it is worth to use this method (p122)
- **Using index or hash** to look for mapping tuples: in nested loop method, if there is suitable access route on I (say B+ tree index), it can be used to substitute sequence scan. It is best when there is cluster index or hash on join attributes.
- **Hash join**: because the join attributes of R and S have the same domain, R and S can be hashed into the same hash file using the same hash function, then R $\bowtie$ S can be computed based on the hash file.

Principles of Database Systems, Xu Lizhen

25

4.5 Recovery 防止故障&故障恢复

4.5.1 Introduction

The main roles of recovery mechanism in DBMS are:

- (1) Reducing the likelihood of failures (prevention)
- (2) Recover from failures (solving)

Restore DB to a consistent state after some failures.

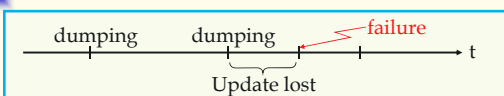
- Redundancy is necessary. 冗余 用于备份
- Should inspect all possible failures.
- General method:

关于几种连接实现算法, 下述说法不正确的是:

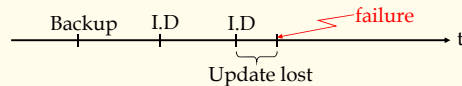
- A 当除了堆文件无其它存取路径可用时, 应采用归并扫描算法
- B 归并扫描算法适合于数据更新很少的应用场景
- C Hash Join需要付出事先创建散列文件的代价
- D 嵌套循环是最基本的、至少可用的连接算法

Principles of Database Systems, Xu Lizhen

1) Periodical dumping 周期备份



- Variation: Backup + Incremental dumping
I.D --- updated parts of DB 备份+增量 仅记录发生变化的部分



This method is easy to be implemented and the overhead is low, but the update maybe lost after failure occurring. So it is often used in file system or small DBMS.

Principles of Database Systems, Xu Lizhen

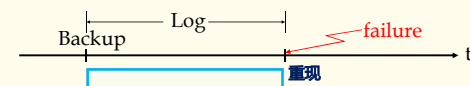
27

2) Backup + Log 日志

Log: record of all changes on DB since the last backup copy was made.

Change: { Old value (before image --- B.I) } Recorded
 { New value (after image --- A.I) } into Log

For update op.: B.I A.I 修改前的值
 insert op.: ---- A.I 修改后的值
 delete op.: B.I ----



Principles of Database Systems, Xu Lizhen

28

While recovering: 对数据库操作的基本单位为: 事务 transactions

- Some transactions maybe half done, should undo them with B.I recorded in Log.
- Some transactions have finished but the results have not been written into DB in time, should redo them with A.I recorded in Log. (finish writing into DB)

It is possible to recover DB to the most recent consistent state with Log.

Principles of Database Systems, Xu Lizhen

29

4.5.2 Transaction ACID 准则 由DBMS 维护

A transaction T is a finite sequence of actions on DB exhibiting the following effects:

- 原子性 ✓ **Atomic action**: Nothing or All.
- 一致保留性 ✓ **Consistency preservation**: consistency state of DB $\rightarrow$ another consistency state of DB.
- 并行不干扰 ✓ **Isolation**: concurrent transactions should run as if they are independent each other.
- 持久性 ✓ **Durability**: The effects of a successfully completed transaction are permanently reflected in DB and recoverable even failure occurs later.

Principles of Database Systems, Xu Lizhen

30

Example: transfer money s from account A to account B

Begin **transaction**

```

read A
A:=A-s
if A<0 then Display "insufficient fund"
           Rollback /*undo and terminate */
else B:=B+s
     Display "transfer complete"
     Commit /*commit the update and terminate */

```

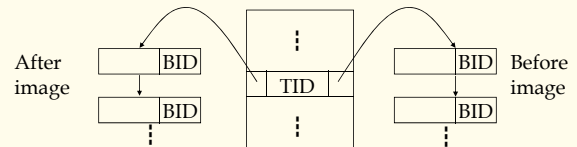
Rollback --- abnormal termination. (Nothing)
Commit --- normal termination. (All)

4.5.3 Some Structures to Support Recovery

非挥发存储
存储器, 断电不消失

Recovery information (such as Log) should be stored in nonvolatile storage. The following information need to be stored in order to support recovery:

- 1) **Commit list**: list of TID which have been committed.
- 2) **Active list**: list of TID which is in progress.
- 3) **Log**:



4.5.4 Commit Rule and Log Ahead Rule

- 1) **Commit Rule** 在事务运行在提交前 要先写下来
A.I must be written to nonvolatile storage before commit of the transaction.
- 2) **Log Ahead Rule** 修改之前需要把原值记入LOG
If A.I is written to DB before commit then B.I must first written to log.
- 3) **Recovery strategies**
(1) The features of undo and redo (are idempotent) :
undo(undo(undo --- undo(x) ---)) = undo(x)
redo(redo(redo --- redo(x) ---)) = redo(x)
利用AI和BI去做undo和redo

幂等

(2) Three kinds of update strategy

多个语句

- a) A.I → DB before commit
TID → active list
B.I → Log (Log Ahead Rule)
A.I → DB
⋮
commit { TID → commit list
delete TID from active list

The recovery after failure in this situation

Check two lists for every TID while restarting after failure: 重新启动恢复

Commit list	Active list	
	✓	Undo, delete TID from active list 故障时事务未完成 还原
✓	✓	delete TID from active list 几乎成功
✓		nothing to do 完全成功

check point 检查点

b) A.I → DB after commit

- TID → active list
A.I → Log (Commit Rule)
⋮
commit { TID → commit list
A.I → DB
delete TID from active list



The recovery after failure in this situation

Check two lists for every TID while restarting after failure:

Commit list	Active list	
	✓	delete TID from active list
✓	✓	redo, delete TID from active list 重新执行AI->DB (需等)
✓		nothing to do



c) A.I → DB **concurrently** with commit

TID → active list

$\left\{ \begin{array}{l} A.I, B.I \rightarrow \text{Log} \\ A.I \rightarrow \text{DB} \end{array} \right.$ (Two Rules)
 (partially done)

⋮

commit $\left\{ \begin{array}{l} TID \rightarrow \text{commit list} \\ A.I \rightarrow \text{DB} \\ \text{delete TID from active list} \end{array} \right.$ (completed)



The recovery after failure in this situation

Check two lists for every TID while restarting after failure:

Commit list	Active list	
	✓	Undo, delete TID from active list
✓	✓	redo, delete TID from active list
✓		nothing to do



Conclusion :

	redo	undo
a)	✗	✓
b)	✓	✗
c)	✓	✓
d)	✗	✗

?

关于检查点 (Check Point), 下述说法正确的是:

- A 检查用户访问数据库的权限
- B 定时设置时间点, 检查事务的执行情况, 完成必要的redo/undo操作
- C 可以提高故障恢复时的处理效率
- D 检查事务中的SQL语句是否存在错误



4.6 Concurrency Control

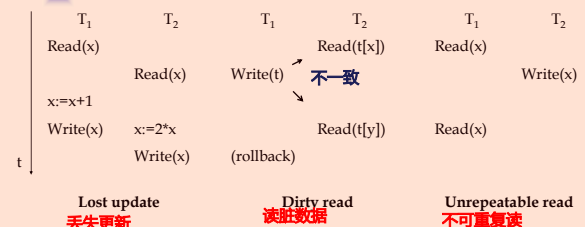
防止并发冲突

4.6.1 Introduction

In multi users DBMS, permit multi transaction access the database concurrently.

Why concurrency?

- 1) Improving system utilization & response time.
 - 2) Different transaction may access to different parts of database.
- Problems arise from concurrent executions



Lost update
丢失更新

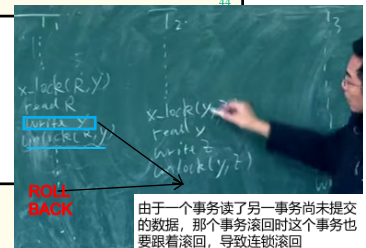
Dirty read
读脏数据

Unrepeatable read
不可重复读

So there maybe three kinds of conflict when transactions execute concurrently. They are write - write, write - read, and read - write conflicts. Write - write conflict must be avoided anytime. Write - read and read - write conflicts should be avoided generally, but they are endurable in some applications.

$\downarrow$ $t \downarrow$	T_A Read R2 Write R2	T_B Read R1 Write R2	T_C Write R1	The result of this schedule is the same as serial execution $T_A \rightarrow T_B \rightarrow T_C$, so it is serializable. The equivalent serial execution is $T_A \rightarrow T_B \rightarrow T_C$.
--	--	--------------------------------------	---------------------------	---

	NL	X
NL	Y	Y
X	Y	N



	T ₁	T ₂
Growing phase	Lock A Lock B Lock C	Lock A Lock B Unlock A Unlock B
Shrinking phase	Unlock A Unlock B Unlock C	Lock C Unlock C
	2PL	not 2PL

	NL	S	X
NL	Y	Y	Y
S	Y	Y	N
X	Y	N	N

	NL	S	U	X
NL	Y	Y	Y	Y
S	Y	Y	Y	N
U	Y	Y	N	N
X	Y	N	N	N

X	(S,X)	(S,U,X)	配合,可得到更好效果
Concurrency degree	→	→	UU 需要排他, 因为一个U结束后需要上升到X并改变数据, 如果不给另外一个U加锁, 它计算的就算旧数据
Overhead	→	→	

Diagram illustrating a deadlock state in a database system:

- Transaction Processes (Left):**
 - T_A : $X_lock R1$, ..., $X_lock R2$, wait
 - T_B : $X_lock R2$, ..., $X_lock R1$, wait
- Resource R (Right):**
 - $T1$: S-lock
 - $T2$: S-lock
 - $T3$: x-lock (等待)
- Annotation:** "又来了S锁" (Another S-lock came) points to $T2$.

- ## 操作系统DNA动了

(1) Deadlock Detection

- 1) Timeout: If a transaction waits for **some specified time** then deadlock is **assumed** and the transaction should be aborted. **依赖于时间设置**
- 2) Detect deadlock by wait-for graph $G=<V,E>$
 V : set of transactions $\{T_i \mid T_i \text{ is a transaction in DBS } (i=1,2,\dots,n)\}$
 E : $\{<T_i,T_j> \mid T_i \text{ waits for } T_j (i \neq j)\}$
 - If there is cycle in the graph, the deadlock occurs.
 - When to detect? **观察等待图**

- 1) **whenever one transaction waits.** **时机: 加入新事务时**
- 2) **periodically** **周期性的**

- What to do when detected?
 - 1) Pick a **victim** (youngest, minimum abort cost, ...)
 - 2) Abort the victim and release its locks and resources
 - 3) Grant a waiter
 - 4) Restart the victim (automatically or manually)

(2) Deadlock avoidance

- 1) Requesting all locks at initial time of transaction.
- 2) Requesting locks in a specified order of resource.
- 3) Abort once conflicted. **排顺序难以实现**
- 4) **Transaction Retry** **使用一个时间戳来标记**
事务重拾 较为可行

死锁? 年老 → 年轻 → 更加年轻 (不会出现循环等待)
 活锁? $\xrightarrow{\text{X (Ta)}} \text{X} \xrightarrow{\text{(t)}}$
 由于比事务X年老是事务只会有有限个

Every transaction **is uniquely time stamped**. If T_A requires a lock on a data object that is already locked by T_B , one of the following methods is used:

- a) **Wait-die**: T_A waits if it is older than T_B , otherwise it "dies", i.e. it is **aborted** and automatically **retried with original timestamp**.
- b) **Wound-wait**: T_A **waits** if it is younger than T_B , otherwise it "wound" T_B , i.e. T_B is aborted and automatically retried with original timestamp.

不讲武德

In above, both have only one direction wait, either older → younger or younger → older. It is impossible to occur wait in cycle, so the dead lock is avoided.

引入时间戳概念, 使等待图中只有年老事务等待年轻事务的单方向等待, 避免出现环路

正确

引入时间戳概念, 使等待图中只有年轻事务等待年老事务的单方向等待, 避免出现环路

它虽然能够避免死锁, 但可能导致活锁

错误

都可以避免活锁!

事务retry的时候更换当前时间戳作为新时间戳也不影响效果

会出现活锁

5. The Security and Integrity Constraints

Introduction

- The destruction of database is generally caused by the following factors:
 - System failure
 - Inconsistency** caused by **concurrent** access
 - Man-caused destruction (intentionally or accidentally)
 - The data inputted is incorrect, the updating transaction didn't obey the rule of consistency preservation
- In above factors, 1 and 2 should be resolved by recovery mechanism of DBMS (Chapter 4); 3 belongs to database security; 4 belongs to integrity constraints

Principles of Database Systems, Xu Lizhen

2

Security of Database

- Protect databases not be accessed illegally.
 - View and query rewriting **视图与查询重写**
 - Access control **访问控制**
 - General user
 - User with resource privilege
 - DBA
 - Identification and authentication of users
 - Password
 - Special articles, such as key, IC card, etc.
 - Personal features, such as fingerprint, signature, etc.
 - Authorization **授权**
 - GRANT** CONNECT TO JOHN IDENTIFIED BY xyzabc;
 - GRANT** SELECT ON TABLE S TO U1 WITH GRANT OPTION;
 - Role **针对角色授权**
 - Data encryption **底层文件加密**
 - Audit trail
 - AUDIT** SELECT, INSERT, DELETE, UPDATE ON emp WHENEVER SUCCESSFUL;

Principles of Database Systems, Xu Lizhen

3

统计数据库的安全

Security of Statistical Database

- In many situation, the statistical data is public while the detailed individual data is secret. **允许对公开数据进行公开查询**
- Public statistical database
- But **some detailed individual data can be derived from public statistical data** **差分隐私?**
 - How prevent this leak? --- not a easy thing

STATS

NAME	SEX	DEPENDENTS	OCCUPATION	SALARY
Wang	M	2	Programmer	120
Chang	F	2	Manager	240
Chen	F	0	Programmer	140
Li	F	2	Engineer	160
Liu	M	2	Clerk	110
Zhu	F	1	Teacher	80
Zhao	M	0	Professor	180
Sun	M	1	Teacher	110
Xu	F	2	Programmer	130
Ma	F	1	Programmer	150

Principles of Database Systems, Xu Lizhen

4

Individual Tracker

- Suppose we know Wang is a male programmer, and salary in STATS is secret but other information is public, we can get wang's salary from public data.

```

> Q1: SELECT COUNT(*)
    FROM STATS
    WHERE SEX='M' AND OCCUPATION='programmer'
result = 1
Q2: SELECT SUM(SALARY)
    FROM STATS
    WHERE SEX='M' AND OCCUPATION='programmer';
result = 120
  
```

Principles of Database Systems, Xu Lizhen

5

Individual Tracker (c>b, b=2)

```

> Q3: SELECT COUNT(*)
    FROM STATS;
result = 10
Q4: SELECT COUNT(*)
    FROM STATS
    WHERE NOT(SEX='M' AND OCCUPATION='programmer');
result = 9
Now we know only one male programmer, that must be Wang.
> Q5: SELECT SUM(SALARY)
    FROM STATS;
result = 1420
Q4: SELECT SUM(SALARY)
    FROM STATS
    WHERE NOT(SEX='M' AND OCCUPATION='programmer');
result = 1300
Wang's salary = Q5 - Q6 = 120
  
```

Principles of Database Systems, Xu Lizhen

6

Individual Tracker ($b < c < n - b$, $b = 2$, n is 10)

- Q7: SELECT COUNT(*)
FROM STATS
WHERE SEX = 'M';
result = 4
- Q8: SELECT COUNT(*)
FROM STATS
WHERE SEX = 'M' AND NOT(OCCUPATION = 'programmer');
result = 3
Now we know only one male programmer, that must be Wang.
- Q9: SELECT SUM(SALARY)
FROM STATS
WHERE SEX = 'M';
result = 520
- Q10: SELECT SUM(SALARY)
FROM STATS
WHERE SEX = 'M' AND NOT(OCCUPATION = 'programmer');
result = 400
Wang's salary = Q9 - Q10 = 120

Principles of Database Systems, Xu Lizhen

7

General Tracker

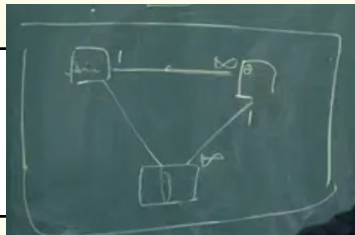
- Individual tracker 个体追踪器
 - Suppose predicate $p = p_1 \text{ and } p_2$. $SET(p)$ is set of tuples which fulfill p , then
 - $SET(p) = SET(p_1 \text{ and } p_2) = SET(p_1) - SET(p_1 \text{ and not } p_2)$
- General tracker 通用追踪器
 - It is a predicate T which fulfill:
 $2b \leq |SET(T)| \leq (n - 2b)$, $b < n/4$ 利用T来追踪任意一个元组
 - Suppose a tuple R can be limited uniquely by predicate p , that is $SET(p) = \{R\}$, then
 $SET(p) = SET(p \text{ or } T) \cup SET(p \text{ or not } T) - SET(T) - SET(\text{not } T)$
 - $\cup$ means union without eliminating repeated tuples.

Principles of Database Systems, Xu Lizhen

8

Integrity Constraints 完整性约束

- An IC describes conditions that every *legal instance* of a relation must satisfy.
 - Inserts/deletes/updates that violate IC's are disallowed.
 - Can be used to ensure application semantics (e.g., *sid* is a key), or prevent inconsistencies (e.g., *sname* has to be a string, *age* must be < 200)



连锁的级联删除

9

Types of Integrity Constraints

- Static constraints: constraints to database state
 - Inherent constraints (data model), such as 1NF 原子的, 不能套表
 - Implicit constraints: implied in data schema, indicated by DDL generally. Such as domain constraints, primary key constraints, foreign key constraints. 域、主键、外键 完整性
 - Domain constraints: Field values must be of right type. Always enforced.
- Dynamic constraints: constraints while database transferring from one state to another. Can be combined with trigger.
 - Explicit constraints or general constraints

Principles of Database Systems, Xu Lizhen

10

Database Modification

- If α is foreign key in r_2 which references to K_1 in r_1 , the following tests must be made in order to preserve the following referential integrity constraint:
 $\Pi_{\alpha}(r_2) \subseteq \Pi_{K_1}(r_1)$
 - Insert. If a tuple t_2 is inserted into r_2 , the system must ensure that there is a tuple t_1 in r_1 such that $t_1[K_1] = t_2[\alpha]$. That is
 $t_2[\alpha] \in \Pi_{K_1}(r_1)$
 - Delete. If a tuple, t_1 is deleted from r_1 , the system must compute the set of tuples in r_2 that reference t_1 :
 $\sigma_{\alpha = t_1[K_1]}(r_2)$ R2中a对应R1的主键K1消失
- If this set is not empty, either the delete command is rejected as an error, or the tuples that reference t_1 must themselves be deleted (cascading deletions are possible).

报错

级联删除

Principles of Database Systems, Xu Lizhen

11

Database Modification (Cont.)

级联删除与...

- Update. There are two cases: 类似于insert t2
 - If a tuple t_2 is updated in relation r_2 and the update modifies values for foreign key α , then a test similar to the insert case is made. Let t_2' denote the new value of tuple t_2 . The system must ensure that
 $t_2'[\alpha] \in \Pi_{K_1}(r_1)$ 类似于insert t1
 - If a tuple t_1 is updated in r_1 , and the update modifies values for the primary key (K_1), then a test similar to the delete case is made. The system must compute
 $\sigma_{\alpha = t_1[K_1]}(r_2)$
using the old value of t_1 (the value before the update is applied). If this set is not empty, the update may be rejected as an error, or the update may be cascaded to the tuples in the set, or the tuples in the set may be deleted.

如果K1是关系r1的主键, α 是关系r2中的外键, 它引用了关系r1的K1, 则在对r1、r2做更新时, 下述说法正确的是:

当更新r2中元组t2的 α 值时, 情形类似于在r2中插入新元组

当更新r1中元组t1的K1值时, 情形类似于在r1中删除元组, 需考虑级联更新

Definition of Integrity Constraints

- Indicated with procedure **在程序中指定**
 - Let application programs responsible for the checking of integrity constraints.
- Indicated with **ASSERTION**
 - Defined with *assertion specification language*, and checked by DBMS automatically
 - ASSERT** balanceCons ON account: balance >= 0;
- Indicated with **CHECK** clause in base table definition, and checked by DBMS automatically

Principles of Database Systems, Xu Lizhen

13

General Constraints

- Useful when more general ICs than keys are involved.
- Can use queries to express constraint.
- Constraints can be named.

```
CREATE TABLE Sailors
(sid INTEGER,
sname CHAR(10),
rating INTEGER,
age REAL,
PRIMARY KEY (sid),
CHECK ( rating >= 1
AND rating <= 10))

CREATE TABLE Reserves
(sname CHAR(10),
bid INTEGER,
day DATE,
PRIMARY KEY (bid, day),
CONSTRAINT noInterlakeRes
CHECK ('Interlake' <>
(SELECT B.bname
FROM Boats B
WHERE B.bid=bid)))
```

create table Teachers(

```
...
Age int check(age > 18 and age < 30),
...
)

create table Teachers(
...
Age int,
...
constraint ck_age check (age > 18 and age < 30)
)
```

14

两个约束作用相同，只不过一个在列级定义，一个在表级定义

Constraints Over Multiple Relations

```
CREATE TABLE Sailors
(sid INTEGER,
sname CHAR(10),
rating INTEGER,
age REAL,
PRIMARY KEY (sid),
CHECK
((SELECT COUNT (S.sid) FROM Sailors S)
+(SELECT COUNT (B.bid) FROM Boats B) < 100))
```

Number of boats
plus number of
sailors is < 100

只会在Sailors表插入、删除、修改时被检查

- Awkward and wrong!
- If Sailors is empty, the number of Boats tuples can be anything!

Principles of Database Systems, Xu Lizhen

15

Assertion 完整性约束规则

- ASSERTION is the right solution; not associated with either table. 涉及多个表的约束 应该使用断言

```
CREATE ASSERTION smallClub
CHECK
((SELECT COUNT (S.sid) FROM Sailors S)
+(SELECT COUNT (B.bid) FROM Boats B) < 100))
```

对于上述两个约束定义，下述说法正确的是：

Principles of Database Systems, Xu Lizhen

16

Triggers 触发器

- Trigger: procedure that starts automatically if specified changes occur to the DBMS
- Three parts:
 - Event (activates the trigger) 什么事件发生 满足条件
 - Condition (tests whether the triggers should run)
 - Action (what happens if the trigger runs) 执行行动
- Active database rules (ECA rules)
 - 主动数据库 规则
 - 让数据库的系统自动维持 某些生产原材的 采购

Principles of Database Systems, Xu Lizhen

17

Triggers: Example (SQL:1999)

```
CREATE TRIGGER youngSailorUpdate
AFTER INSERT ON SAILORS
REFERENCING NEW TABLE NewSailors
FOR EACH STATEMENT
INSERT
    INTO YoungSailors(sid, name, age, rating)
    SELECT sid, name, age, rating
    FROM NewSailors N
    WHERE N.age <= 18
```

Principles of Database Systems, Xu Lizhen

18



Execution of Rules

- Immediate execution ✓
- Deferred execution 延迟执行 (在事务完全执行完毕了以后)
- Decoupled or detached mode 建立事务的派生关系
解耦与分离的模式
- Cascading trigger
 - Control nested execution of rules 连锁触发
 - Prevent nontermination
 - Triggering graph 构造触发图
 - Specify the upper limit of cascading times
 - So triggers should be used reasonably

```
CREATE TRIGGER 不能再订绿船
BEFORE INSERT ON Reserves
REFERENCING NEW AS N
FOR EACH ROW
WHEN (N.bid IN (SELECT bid FROM Boats WHERE
               color = 'Green') AND
      N.sid IN (SELECT sid FROM Reserves, Boats
               WHERE Reserves.bid=Boats.bid AND color='Red'))
ROLLBACK;
```

对于上述触发器，下面说法正确的是：

- ☒ A 它的事件 (E) 是对Reserves表做INSERT之前
- ☒ B 它引用的是INSERT操作的新值
- ☐ C 它的条件 (C) 是检查新预定的船是否为绿船
- ☒ D 它的动作 (A) 是滚回，拒绝此次插入操作

19



Implementation of ECA

- Loosely coupling 在原来不支持的内核上添加插件 拦截符合条件的语句 松耦合
 - Tightly coupling (DB2, Oracle, etc.) 需要重新设计内核 彻底的实现
 - Nested method 嵌套的方法 有局限
- The rules are nested into transaction and executed by DBMS as a part of the transaction.
- Grafting method 在sql内核之上 嫁接
 - Query modification method 查询重写 改写 耦合如查询中

```
CREATE TRIGGER <TriggerName>
(before|after ) {insert|delete|update}
ON <RelationName>
[referencing old as <OldName>, new as <NewName>]
[WHEN <Condition>]
[for each row|statement]
<SQLStatement>
```

20

关于触发器的级联触发问题，下面说法正确的是：

- ☐ A 它是由与引用完整性相关的级联更新造成的
- ☐ B 它是指一个触发器触发了多个动作 (A)
- ☒ C 由于一个触发器的动作导致另一个触发器的事件，而且条件满足，从而连续触发多个触发器
- ☐ D 触发器的定义必须慎重，团队开发时不能任由每个成员随意定义触发器

可以采用松耦合方式，在已有DBMS核心之上包一层，在一定程度上扩展对触发器的支持

可以采用紧耦合的方法，从DBMS设计之初就考虑对触发器的支持

可以采用嵌入的方法，适当修改DBM的事务执行模块，扩展一些类似触发器的功能

松耦合和嵌入的方法都属于打补丁的策略，不能实现完整的触发器功能

6. Database Design*

6.1 Data Dependency and Normalization of Relational Schema 数据依赖关系

- Some dependent relations exist between attributes.
- Function dependency (FD):** the most basic kind of data dependencies. The value of one or a group attributes can decide the value of other attributes. FD is the most important in general database design.
 - 函数依赖** 学号→姓名
- Multi-valued Dependency (MVD):** the value of some attribute can decide a group of values of some other attributes. 删除多值依赖得到4NF
 - 多值依赖** 人→语文 数学
- Join Dependency (JD):** the constraint of lossless join decomposition. 删除连接依赖得到5NF
 - 连接依赖** 供应商-零件-工程 三元关系 拆开以后重新连接可以还原

Principles of Database Systems, Xu Lizhen

2

1NF

every attribute of a relation must be atomic.

name	dept	address		
		prov	city	street

Non 1NF

name	dept	prov	city	street
------	------	------	------	--------

1NF

Principles of Database Systems, Xu Lizhen

3

2NF

- $R \in 1NF$ and no partially function dependency exists between attributes.

不存在属性对主键的部分函数依赖

$S(S\#, \underline{SNAME}, AGE, ADDR, \underline{C\#}, \underline{GRADE})$

--- non 2NF

主键: $(S\# C\#) \rightarrow GRADE$

但是 $(S\#) \rightarrow SNAME AGE ADDR$ 对主键的部分函数依赖

Principles of Database Systems, Xu Lizhen

4

Problems of non 2NF:

- Insert abnormality:** can not insert the students' information who have not selected course. 没课程也无法加信息
- Delete abnormality:** if a student unselect all courses, his basic information is also lost. 课程全没了就丢了
- Hard to update:** because of redundancy, it is hard to keep consistency when update. 大量数据冗余

一事一地: 一张表管一个事情

Resolving:

According to the rule of "one fact in one place" to **decompose** the relation into 2 new relations:

$S(S\#, SNAME, AGE, ADDR)$

$SC(S\#, C\#, GRADE)$

Principles of Database Systems, Xu Lizhen

5

3NF

- $R \in 2NF$ and no transfer function dependency exists between attributes.

不存在组件对属性的传递依赖

$EMP(EMP\#, SAL_LEVEL, SALARY)$

--- non 3NF

EMP# 决定了LV
LV决定了SALARY

这里EMP#为主键
 $EMP\# \rightarrow LV SALARY$
所以满足2NF

Principles of Database Systems, Xu Lizhen

6

Problems of non 3NF LV > SALARY 的标准不独立

- ✓ **Insert abnormality**: before the employees's sal_level are decided, the correspondence between sal_level and salary can not input.
- ✓ **Delete abnormality**: if some sal_level has only one man, the correspondence between sal_level and salary of this level will be lost when the man is deleted.
- ✓ **Hard to update**: because of redundancy, it is hard to keep consistency when update.

Resolving:

According to the rule of **"one fact in one place"** to decompose the relation into 2 new relations:

EMP(EMP#,SAL_LEVEL)

SAL(SAL_LEVEL,SALARY)

Principles of Database Systems, Xu Lizhen

7

Material Card

How to define relations to express the information on this card?

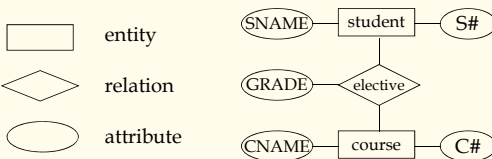
Equipment name:		Type:	Code:				
Unit price:		Store place:	room	rack	layer	position	
date	voucher No.	coming/going place	take in	take out	balance	sum	remark

Principles of Database Systems, Xu Lizhen

8

5.2 ER Model and ER Diagram

- Concept model: entity – relation, be independent of practical DBMS.
- Legend:



Principles of Database Systems, Xu Lizhen

9

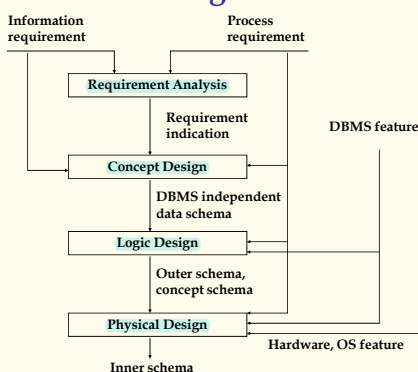
5.3 Database Design Method

- **Procedure oriented method** 过程为中心 没有分析数据的内在关系
This method takes business procedures as center, the database schema is designed basically in accordance directly with the vouchers, receipts, reports, etc. in business. Because of no detailed analysis on data and inner relationships between data, although it is fast at the beginning of the project, it is hard to ensure software quality and the system will be hard to fit future changes in requirement and environment. So this method is not suitable for the development of a large, complex system.
- **Data oriented method** 以数据为中心 流程过程数据抓取
This method design the database schema based on the detailed analysis on data and inner relationships between data which are involved in business procedures. It takes data as center, not procedures. It can not only fulfill the current requirements, but also some potential requirements. It is liable to fit future changes in requirement and environment. It is recommended in the development of large, complex systems.

Principles of Database Systems, Xu Lizhen

10

Database Design Flow



Principles of Database Systems, Xu Lizhen

11

Requirement Analysis

A very important part of system requirement analysis. In requirement analysis phase, the data dictionary and DFD (or UML) diagrams are the most important to database design.

- **Dictionary and DFD**
 - Name conflicts **命名冲突**
 - Homonym (the same name with different meanings)
 - Synonym (the same meaning in different names)
 - Concept conflicts **概念冲突 属性与实体**
 - Domain conflicts **域冲突**
- **About coding** 信息标准化
 - Standardization of information
 - Identifying entities
 - Compressing information
- **Through requirement analysis, all information must be with unique source and unique responsibility.**

Principles of Database Systems, Xu Lizhen

12



Concept Design

Based on data dictionary and DFD, analyze and classify the data in data dictionary, and refer to the processing requirement reflected in DFD, identify entities, attributes, and relationships between entities. Then we can get concept schema of the database.

- Identify Entities
- Define the relationships between entities
- Draw ER diagram and discuss it with user
 - It is proposed to use ER design tools such as ERWin, Rose, etc.



Logic Design

According to the entities and relationships in ER diagram, define tables and views in target DBMS. Basic standard is 3NF.

- Translate entities and relationships in ER diagram to tables
- Naming rule of table and attribute
- Define the type and domain of every attribute
- Suitable denormalization 适当的逆规范化 减少查询连接
- Necessary view
- Consider the tables in legacy system 遗留系统
- Interface tables



Physical Design

For relational database, the main task in this phase is to consider creating necessary indexes according to the processing requirements, including single attribute indexes, multi attributes indexes, cluster indexes, etc. Generally, the attribute often as query conditions should have index.

Other problems:

- Partition design 切割表为不同物理分区 水平分区和垂直分区
- Stored procedure 存取路径
- Trigger 触发器
- Integrity constraints 完整性约束



Remarks

- 仅仅在结构上达到3NF (BCNF) 是不够的。
- “一事一地”包括每项信息的唯一，要提取出问题的本质，识别出本质上同一概念的信息项。
- 对于表达类似信息，模式相似只是取值不同的表，应尽量合并。如学习经历、进修经历；奖励信息、惩处信息等。
- 考虑到效率、用途等因素，该分开的表还应分开。如本科生基本信息和研究生基本信息。
- 结合DBMS内部实现技术，合理设计索引和文件结构，为查询优化准备好存取路径。
- 在结构规范化、减少数据冗余和提高数据库访问性能之间仔细权衡，适当折中。

数据库设计实例分析

7. Distributed Database Systems

7.1 Introduction

What is DDB?

A DDB is a **collection of correlated data** which are spread across a network and managed by a software called DDBMS. **分布在网络上由DDBMS管理**

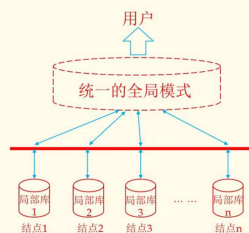
Two kinds:

- (1) Distributed physically, **centralized logically** (general DDB)
- (2) Distributed physically, distributed logically too (FDDBS)
联邦数据库

We take the first as main topic in this course.

Features of DDBS :

- Distribution
- Correlation
- DDBMS



The advantages of DDBS:

- Local autonomy **局部自治**
- Good availability (because support **multi copies**)
- Good **flexibility**
- Low system cost **最好让库本地解决问题**
- High **efficiency** (**most access processed locally**, less communication comparing to centralized database system)
- Parallel process

<10%的标准

The disadvantages of DDBS:

- Hard to **integrate** existing databases **基本要从来来过**
- **Too complex** (system itself and its using, maintenance, etc. such as DDB design) **考虑全局分布 建设 要求非常高**
由于要求要有统一的全局模式，不适合做多数据库集成

The main problems in DDBS:

Compared to centralized DBMS, the differences of DDBS are as follows:

- **Query Optimization** (different optimizing goal) **网络速度导致的瓶颈**
- **Concurrency control** (should consider whole network) **并发操作**
- **Recovery mechanism** (all sub-transactions must commit or abort simultaneously)

Another problem specially for DDBS:

- Data distribution

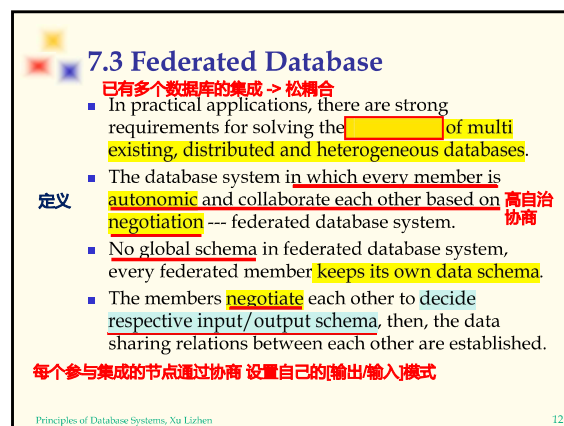
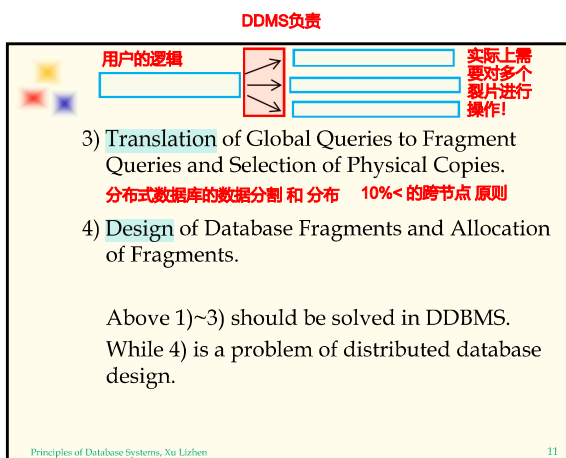
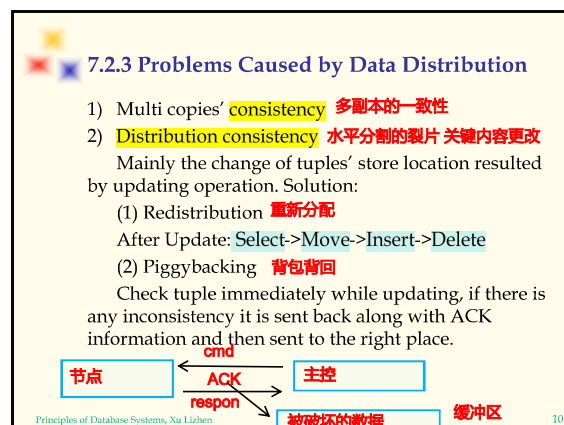
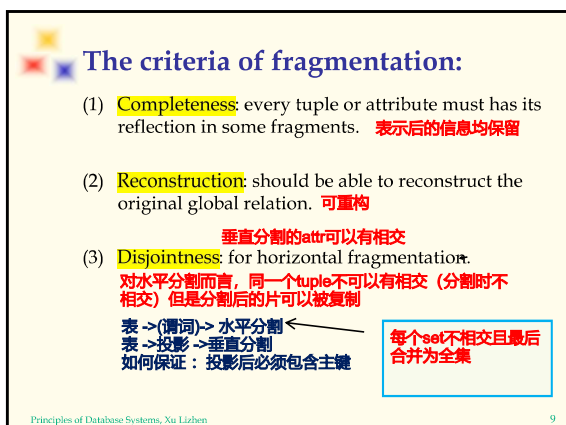
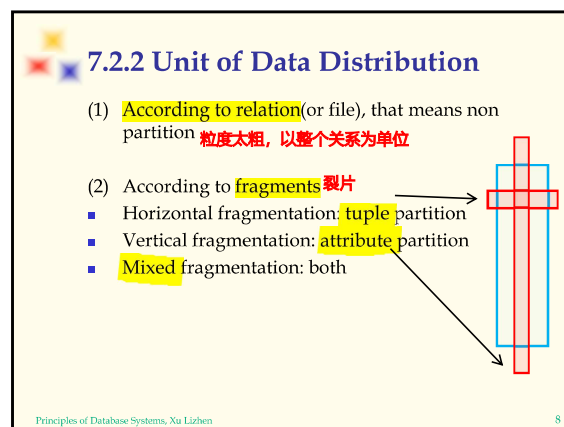
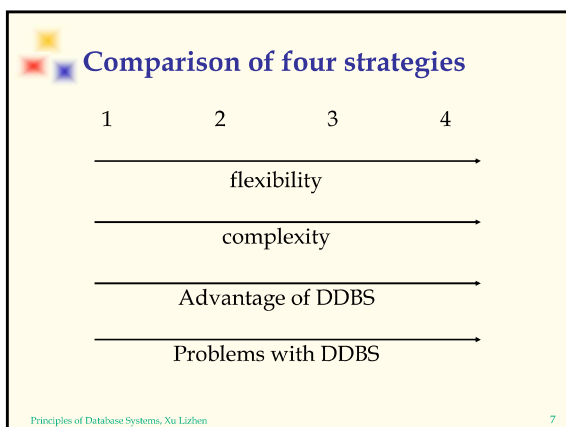
7.2 Data Distribution

拥有统一全局模式，透明

7.2.1 Strategies of Data Distribution

- (1) **Centralized**: distributed system, but the data are still **stored centralized**. It is simplest, but there is not any advantage of DDB.
- (2) **Partitioned**: data are distributed without repetition. (**no copies**) **划分**
- (3) **Replicated**: a **complete copy** of DB at each site. Good for retrieval-intensive system.
- (4) **Hybrid** (mix of the above): an arbitrary fraction of DB at various sites. The most flexible and complex distributing method.

每个节点都是副本



8. New Research and Application Fields

Main Contents

- Data warehouse
- OLAP
- Data mining
- Information retrieval
- Semistructured data and XML

8.2 Data Mining

Definition:

Data mining is the exploration and analysis of large quantities of data in order to discover valid, novel, potentially useful, and ultimately understandable patterns in data.

Example pattern (Census Bureau Data):
If (relationship = husband), then (gender = male). 99.6%

Definition (Cont.)

Data mining is the exploration and analysis of large quantities of data in order to discover valid, novel, potentially useful, and ultimately understandable patterns in data.

- **Valid:** The patterns hold in general.
- **Novel:** We did not know the pattern beforehand.
- **Useful:** We can devise actions from the patterns.
- **Understandable:** We can interpret and comprehend the patterns.

Why Use Data Mining Today?

Human analysis skills are inadequate:

- Volume and dimensionality of the data
- High data growth rate

Availability of:

- Data
- Storage
- Computational power
- Off-the-shelf software
- Expertise

An Abundance of Data

- Supermarket scanners, POS data
- Preferred customer cards
- Credit card transactions
- Direct mail response
- Call center records
- ATM machines
- Demographic data
- Sensor networks
- Cameras
- Web server logs
- Customer web site trails



Much Commercial Support

- Many data mining tools
 - <http://www.kdnuggets.com/software>
- Database systems with data mining support
- Visualization tools
- Data mining process support
- Consultants



Why Use Data Mining Today?

Competitive pressure!

"The secret of success is to know something that nobody else knows."

Aristotle Onassis

- Competition on service, not only on price (Banks, phone companies, hotel chains, rental car companies)
- Personalization, CRM
- The real-time enterprise
- "Systemic listening"
- Security, homeland defense



Data Mining is Supported by Three Sufficiently Mature Technologies

- **Massive data collections**
Commercial databases (using high performance engines) are growing at exceptional rates
- **Powerful multiprocessor computers**
cost-effective parallel multiprocessor computer technology
- **Data mining algorithms**
under development for decades, in research areas such as statistics, artificial intelligence, and machine learning, but now implemented as mature, reliable, understandable tools that consistently **outperform** older statistical methods



Applications, Operations, and Techniques

Applications	Database Marketing Customer Segmentation Customer Retention Fraud Detection Credit Checking Web Site Analysis, etc.
Operations	Classification and Prediction Clustering Association Analysis Forecasting
Techniques	Cluster Analysis Nearest Neighbour Neural Networks Naïve-Bayes Decision Trees



The Knowledge Discovery Process

Steps:

1. Identify business problem
2. Data mining
3. Action
4. Evaluation and measurement
5. Deployment and integration into businesses processes



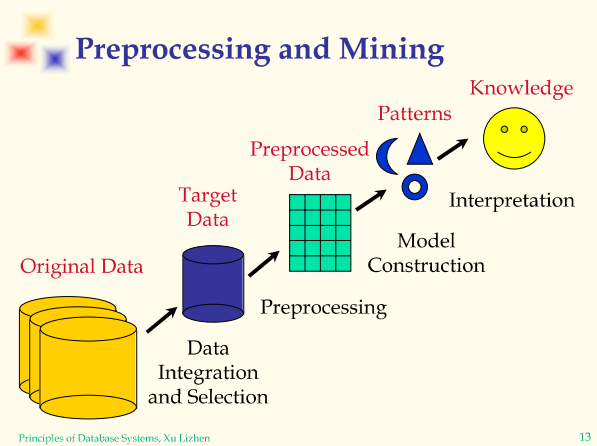
Data Mining Step in Detail

2.1 Data preprocessing

- Data selection: Identify target datasets and relevant fields
- Data cleaning
 - Remove noise and outliers
 - Data transformation
 - Create common units
 - Generate new fields

2.2 Data mining model construction

2.3 Model evaluation



Example Application: Sky Survey

- Input data: 3 TB of image data with 2 billion sky objects, took more than six years to complete
- Goal: Generate a catalog with all objects and their type
- Method: Use decision trees as data mining model
- Results:
 - 94% accuracy in predicting sky object classes
 - Increased number of faint objects classified by 300%
 - Helped team of astronomers to discover 16 new high red-shift quasars in one order of magnitude less observation time

Principles of Database Systems, Xu Lizhen

Example Application: Market Baskets

- An important form of mining involves *market baskets* = sets of "items" that are purchased together as a customer leaves a store.
- Summary of basket data is *frequent itemsets* = sets of items that often appear together in baskets.
- If people often buy hamburger and ketchup together, the store can:
 1. Put hamburger and ketchup near each other and put potato chips between.
 2. Run a sale on hamburger and raise the price of ketchup.

Principles of Database Systems, Xu Lizhen

Data Mining: Types of Data

- Relational data and transactional data
- Spatial and temporal data, spatial-temporal observations
- Time-series data
- Text
- Images, video
- Mixtures of data
- Sequence data
- Features from processing other data sources

Principles of Database Systems, Xu Lizhen

Types of Variables

- *Numerical*: Domain is ordered and can be represented on the real line (e.g., age, income)
- *Nominal* or *categorical*: Domain is a finite set without any natural ordering (e.g., occupation, marital status, race)
- *Ordinal*: Domain is ordered, but absolute differences between values is unknown (e.g., preference scale, severity of an injury)

Principles of Database Systems, Xu Lizhen

Data Mining Operations

Each operation (or task) reflects a different way of distinguishing patterns or trends in complex datasets

- Classification and prediction
- Clustering
- Association analysis and sequential analysis
- Forecasting

Principles of Database Systems, Xu Lizhen



Classification and Prediction

Classification is the operation most commonly supported by commercial data mining tools

- It is the process of sub-dividing a data set with regard to a number of specific outcomes.
 - For example, classifying customers into 'high' and 'low' categories with regard to credit risk.
- The category or 'class' into which each customer is placed is the 'outcome' of the classification.



Techniques for Classification and Prediction

- Decision trees
- Neural networks
- Nearest neighbour algorithms



Understanding v Prediction

- Sophisticated classification techniques enable us to discover new patterns in large and complex data sets.
- Classification is a powerful aid to understanding a particular problem.
- In some cases, improved understanding is sufficient. It may suggest new initiatives and provide information that improves future decision making.
- Often the reason for developing an accurate classification model is to improve our capability for prediction.



Training

- A classification model is said to be 'trained' on historical data, for which the outcome is known for each record.
- But beware overfitting: 100 per cent of customers called Smith who live at 28 Arcadia Street responded to our offer.
- One would then use a separate test dataset of historical data to validate the model.
- The model could then be applied to a new, unclassified data set in order to predict the outcome for each record.



Clustering

- Clustering is an unsupervised operation - no training.
- It is used to find groupings of similar records in a data set without any preconditions as to what that similarity may involve.
- Clustering is used to identify interesting groups in a customer base that may not have been recognised before.
- Often undertaken as an exploratory exercise before doing further data mining using a classification technique.



Techniques for Clustering

- Cluster analysis
- Neural networks
- + Good visualization support for "playing" with clusters, to see if they make sense and are useful in a business context



Association Analysis

- **Association Rule** looks for links between records in a data set.
Sometimes referred to as 'market basket analysis', its most common aim is to discover which items are generally purchased at the same time.
- **Time Sequential** looks for temporal links between purchases, rather than relationships between items in a single transaction.



Example of Association Rule

- Consider the following beer and nappy example:
 - 500,000 transactions
 - 20,000 transactions contain nappies (4%)
 - 30,000 transactions contain beer (6%)
 - 10,000 transactions contain both nappies and beer (2%)



Support (or prevalence)

- Measures how often items occur together, as a percentage of the total transactions.
- In this example, beer and nappies occur together 2% of the time (10,000/500,000).



Confidence (or predictability)

- Measures how much a particular item is dependent on another.
- Because 20,000 transactions contain nappies and 10,000 of these transactions contain beer, when people buy nappies, they also buy beer 50% of the time.
- The confidence for the rule:
When people buy nappies they also buy beer 50% of the time.
is 50%.



Confidence (cont)

- Because 30,000 transactions contain beer and 10,000 of these transactions contain nappies, when people buy beer, they also buy nappies 33.33% of the time.
- The confidence for the rule:
When people buy beer they also buy nappies 1/3 of the time.
is 33.33%.
- Both these rules have the same support: 2%



Expected Confidence

- In the absence of any knowledge about what else was bought, we can also make the following assertions from the available data:
People buy nappies 4% of the time.
People buy beer 6% of the time.
- These numbers - 4% and 6% - are called the **expected confidence** of buying nappies or beer, regardless of what else is purchased.



Lift

- Measures the ratio between the confidence of a rule and the expected confidence that the second product will be purchased. Lift is measures of the strength of an effect.
- In our example, the confidence of the nappies-beer buying rule is 50%, whilst the expected confidence is 6% that an arbitrary customer will buy beer.
- So, the lift provided by the nappies-beer rule is: $8.33 (= 50\% / 6\%)$.
- A key goal of an association analysis is to find rules that have a substantial lift like this.



Forecasting

- Forecasting (unlike prediction based on classification models) concerns the prediction of continuous values, such a person's income based on various personal details, or the level of the stock market.
- Simpler forecasting problems involve a single continuous value based on a series of unordered examples.
- More complex problem is to predict one or more values based on a sequential pattern.
- Techniques include statistical time-series analysis as well as neural networks.



Data Mining Techniques

Each technique is used to support a particular data mining operation

- Cluster Analysis
- Association Rules Algorithm
- Time Sequence Algorithm
- Decision Trees



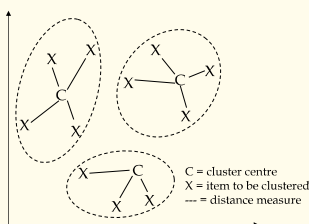
Cluster Analysis

- Identifies the relationships that exist between items on the basis of their similarity and dissimilarity.
- Clusters are typically based around a "centre" value.
- How centres are initially defined and adjusted varies between algorithms.
- One method is to start with a random set of centres, which are then adjusted, added to and removed as the analysis progresses.



Cluster Analysis (cont)

- To identify items that belong to a cluster, some measure must be used that gauges the similarity between items within a cluster and their dissimilarity to items in other clusters.



- This is typically measured as their distance from each other and from the cluster centres within a multi-dimensional space, where each dimension represents one of the variables being compared.

- The key is the design and implementation of distance function.



Association Rule Algorithm

- Apriori algorithm (R.Agrawal, etc.)
 1. Find out all frequent itemsets
 2. Generate candidate association rules based on frequent itemsets and verify their confidence.

```
Procedure AprioriAlg()
Begin
  L1 := {frequent 1-itemsets}; /* 1-itemsets is itemsets with single item */
  for(k=2; L_{k-1} ≠ Φ; k++) {
    C_k := AprioriGen(L_{k-1}); /* candidate itemset generated by AprioriGen */
    for all transactions in the dataset do
      {for all candidates c ∈ C_k contained in t do
        c.count++;
      }
    L_k := {c ∈ C_k | c.count ≥ min-support}
  }
  Answer := ∪_k L_k;
end
```



Time Sequence Algorithm

- AprioriALL algorithm
- 1. Sort (according to customer ID and then transaction time)
- 2. Find out frequent itemset (similar as Apriori, but count according to customer)
- 3. Transform customer transaction sequence
- 4. Find out frequent sequences (similar as Apriori, but AprioriGen() is some different)
- 5. Select maximum frequent sequences, they are the answer.



Classification Algorithm

- The generation and maintenance algorithm of decision tree

```
partition(S)
  if(all tuples in S are of the same class ) then
    return; /*don't need to classify */
  foreach value of attribute A do
    evaluate the split on that value;
  use best split found to partition S into S1 and S2;
  partition S1; /* call partition() recursively */
  partition S2;

■ Evaluate each split:
Gini(S) = 1 -  $\sum_{j=1}^n p_j^2$  /* n is class number in S; pj is frequency of every class in S */

Ginisplit(S) = (N1/N)*gini(S1) + (N2/N)*gini(S2)

Select smallest Ginisplit(S) as split condition.
```

8. New Research and Application Fields

比方说超市购物订单
数据中的数量非常非常大，属于企业资产

Main Contents

- Data warehouse
- OLAP
- Data mining
- Information retrieval
- Semi-structured data and XML

	OLTP System	OLAP System
	Online Transaction Processing	Online Analytical Processing
业务目的	处理业务，如订单、合同等	业务支持决策
面向对象	业务处理人员	分析决策人员
主要工作负载	增、删、改	查询
主要衡量指标	事务吞吐量	查询响应速度（QPS）
数据库设计	3NF 或 BCNF	星型/雪花模型

8.1 Data Warehouse & OLAP

- Challenges come from huge amount of data in network era
 - How to use them efficiently?
 - How to find useful information in such a data ocean?
 - If they cannot be used by human being, they are garbage.
- Data is the most important resources.
- The importance of decision-making scientifically
- Data requirements in decision-making

Principles of Database Systems, Xu Lizhen

3

Data Requirements in Decision-making

- Need summarized data while not detail data
OLTP才需要记录
- Need historical data
- Need large amount of external data (multi data source)
面向决策主题 多源的外部数据
- Need decision subject oriented data, while not the data facing daily transaction process
- The data don't need real time updating. They are mainly read
- OLTP and OLAP

Principles of Database Systems, Xu Lizhen

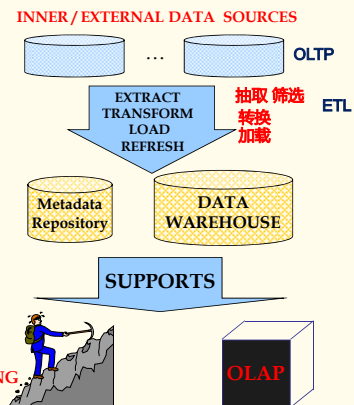
4

What is Data warehouse

- A data warehouse is a repository of information gathered from multiple sources.
 - Decision subject oriented 面向决策主题
 - Provides a single consolidated interface to data 数据集成
 - Data stored for an extended period, providing access to historical data
 - Mainly retrieved 主要是查询
 - Data/updates are periodically downloaded from online transaction processing (OLTP) systems. 周期更新
 - Typically, download happens each night.
 - Data may not be completely up-to-date, but is recent enough for analysis.
 - Running large queries at the warehouse ensures that OLTP systems are not affected by the decision-support workload.

Principles of Database Systems, Xu Lizhen

5



Principles of Database Systems, Xu Lizhen

6

Database and Data Warehouse

	On-Line Transaction Processing (OLTP)	On-Line Analytical Processing (OLAP)
Data Feature	Detail data	Summarized data
Data prescription	Current data	Current and historical data
Data Source	Inner data of an enterprise	Inner & External data; Distributed; Heterogeneous
Data organization	Surrounding transaction processing	Surrounding decision subject
Data Updating	Updated instantly	Periodical or on demand
Data Amount	Involving less data in one operation	Involving huge amount of data in one operation
Operating Feature	Mainly simple and repeated short transactions	Mainly long transaction processing complex queries

Principles of Database Systems, Xu Lizhen 7

Warehousing Issues

语义集成

- Semantic Integration:** When getting data from multiple sources, must eliminate mismatches, e.g., different currencies, schemas.
- 异构数据源**
- Heterogeneous Sources:** Must access data from a variety of source formats and repositories.
 - Replication capabilities can be exploited here.
- Load, Refresh, Purge:** Must load data, periodically refresh it, and purge too-old data.
- 数据转化的过程**
- Metadata Management:** Must keep track of source, loading time, and other information for all data in the warehouse.

Principles of Database Systems, Xu Lizhen 8

Software Solutions that use Warehouse

- Online Analytical Processing (OLAP)**
 - enables users to analyses data across multiple dimensions and hierarchies
- Analysis and Query Reporting Solutions**
 - custom built analysis tools use mathematical models to produce specialized interactive solutions
- Data Mining**
 - enable users to identify patterns and correlations within a set of data, or to create predictive models from the data

Principles of Database Systems, Xu Lizhen 9

An Example

Color	Size	Number
Light	small	3
Light	Medium	20
Light	Large	10
Dark	Small	4
Light	Small	5
Dark	Small	16
Light	Medium	5
Dark	Large	5
Light	Medium	10
Dark	Medium	5
Dark	Medium	5

	Small	Medium	Large	Total
Light	8	35	10	53
Dark	20	10	5	35
Total	28	45	15	88

Cross-tabulation of *number* by *size* and *color* of sample relation *sales* with the schema *Sales*(color, size, number).

Principles of Database Systems, Xu Lizhen 10

An Example (Cont.)

Color	Size	Number
Light	Small	8
Light	Medium	35
Light	Large	10
Light	all	53
Dark	Small	20
Dark	Medium	10
Dark	Large	5
Dark	all	35
all	Small	28
all	Medium	45
all	Large	15
all	all	88

- Can represent subtotals in relational form by using the value all
- E.g. : obtain (Light, all, 53) and (Dark, all, 35) by aggregating individual tuples with different values for size for each color.

Principles of Database Systems, Xu Lizhen 11

Multidimensional Data Model 多维统计分析

- Collection of numeric **measures**, which depend on a set of **dimensions**.
 - E.g., measure **Sales**, dimensions **Product** (key: pid), **Location** (locid), and **Time** (timeid).

Slice locid=1 is shown:

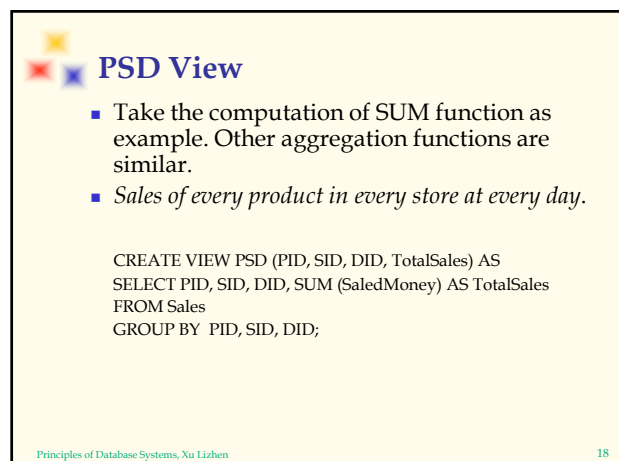
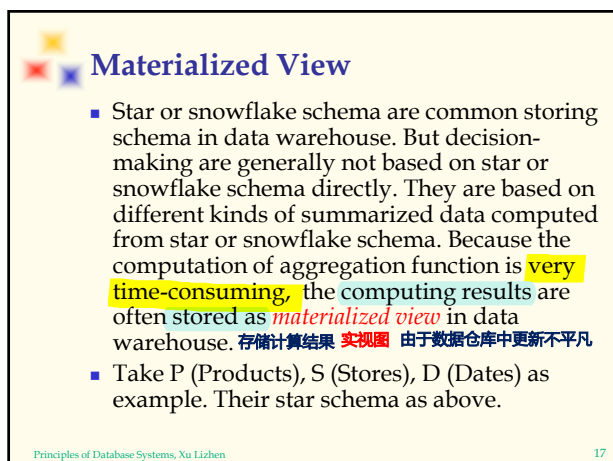
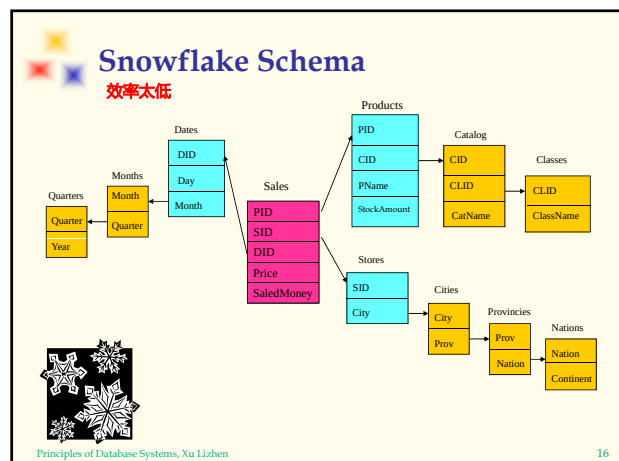
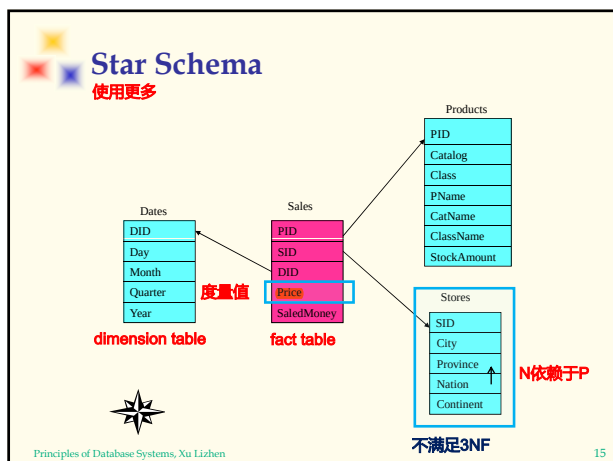
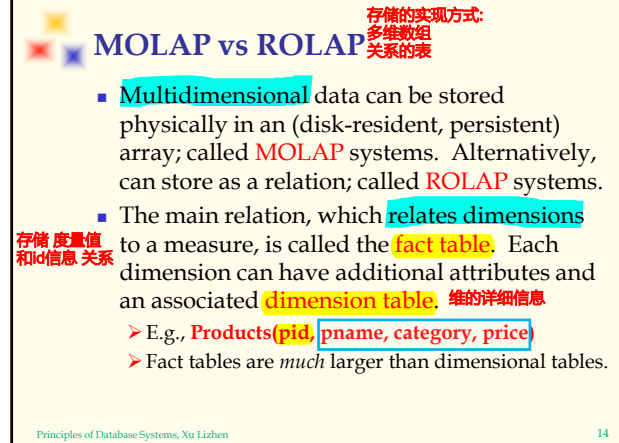
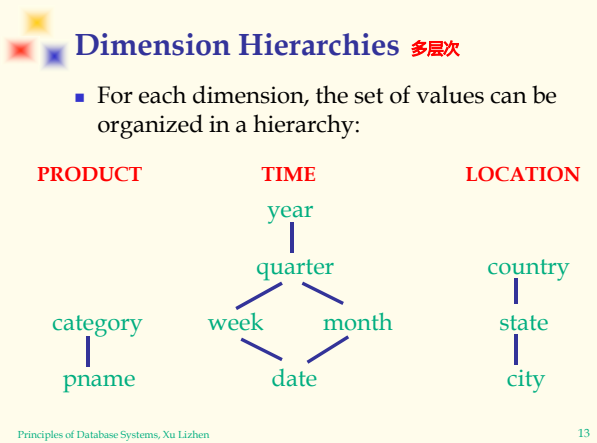
pid: 11, 12, 13

timeid: 1, 2, 3

8	10	10
30	20	50
25	8	15

pid	timeid	locid	sales
11	1	1	25
11	2	1	8
11	3	1	15
12	1	1	30
12	2	1	20
12	3	1	50
13	1	1	8
13	2	1	10
13	3	1	10
11	1	2	35

Principles of Database Systems, Xu Lizhen 12





PS, SD, and PD View

- Total sales of every product in every store (for all times)
- SD and PD views are similar (for all products or for all stores)

```
CREATE VIEW PS (PID, SID, TotalSales) AS
SELECT PID, SID, SUM(TotalSales) AS TotalSales
FROM PSD
GROUP BY PID, SID;
```

- PS View can be expressed as PS ALL



P, S, and D View

- Total sales of every product (for all stores and all times)
- S and D views are similar (aggregated according to store or date respectively)

```
CREATE VIEW P (PID, TotalSales) AS
SELECT PID, SUM(TotalSales) AS TotalSales
FROM PS /* or PD */
GROUP BY PID;
```

- P View can be expressed as P ALL ALL



ALL View

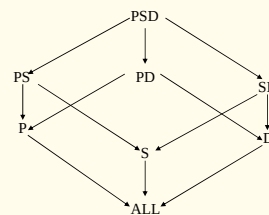
- Total sales for all products and all stores and all times

```
CREATE VIEW ALL (TotalSales) AS
SELECT SUM(TotalSales) AS TotalSales
FROM P /* or S or D */
```

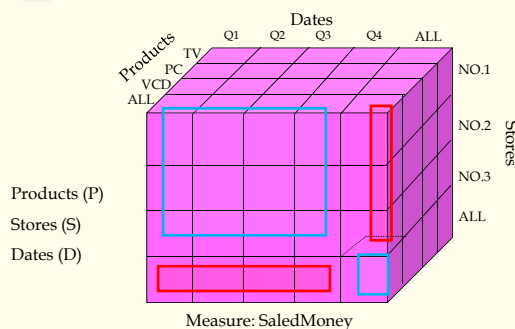


Reliant Relationships Between Views

- There are three dimensions in Sales: {P, S, D}, every sub-set of it is corresponding to a view. These are equivalent to 2^3 elements of power-set $\rho(\{P, S, D\})$.
- The following is the reliant relationships between these materialized views:



A Data Cube Example



OLAP Queries

- A common operation is to **aggregate** a measure over one or more dimensions.
 - Find total sales.
 - Find total sales for each city, or for each state.
 - Find top five products ranked by total sales.
- **Roll-up**: Aggregating at different levels of a dimension hierarchy. 增高粒度 月→年
 - E.g., Given total sales by **city**, we can roll-up to get sales by **state**.

上卷



OLAP Queries

- 下钻**
 - **Drill-down:** The inverse of roll-up.
 - E.g., Given total sales by state, can drill-down to get total sales by city.
 - E.g., Can also drill-down on different dimension to get total sales by product for each state.
- 选择**
 - **Pivoting:** Aggregation on selected dimensions. **选择要看的维度**
- 切片**
 - **Slicing and Dicing:** Equality and range selections on one or more dimensions.



The CUBE Operator

- Generalizing the previous example, if there are k dimensions, we have 2^k possible SQL GROUP BY queries that can be generated through pivoting on a subset of dimensions.
- **CUBE PID, SID, DID BY SUM Sales** 如同之前的8个视图生成
 - Equivalent to rolling up Sales on all eight subsets of the set {pid, locid, timeid}; each roll-up corresponds to an SQL query of the form:

Lots of work on optimizing the CUBE operator!

```
SELECT SUM (S.sales)
FROM   Sales S
GROUP BY grouping-list
```



Examples of Queries on Cube

```
SELECT PID, SID, Quarter, SUM (SaledMoney) AS TotalSales
FROM Sales S, Dates D
WHERE S.DID=D.DID
CUBE BY PID, SID, Quarter;
--- Generate a data cube including  $2^3$  views.
```

- CUBE(TV, No1, Q1)
 - Query TV's total sales of store No1 in first quarter.
- CUBE(ALL, No1, Q1)
 - Query total sales of store No1 in first quarter.
- CUBE(ALL, ALL, Q1)
 - Query total sales in first quarter.
- CUBE(ALL, ALL, ALL)
 - Query total sales in whole year.
- CUBE(TV, No1, Q1) + CUBE(VCD, No1, Q1)
- CUBE(ALL, ALL, Q1) / CUBE(ALL, ALL, ALL) * 100%



Examples of Roll-up / Drill-down

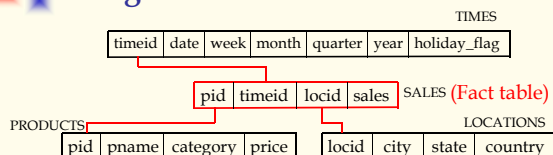
```
SELECT PID, SID, DID, SUM (SaledMoney) AS TotalSales
FROM Sales
GROUP BY PID, SID, DID ROLLUP;
--- Generate four views : PSD, PS, P, and ALL
```

```
SELECT PID, SID, DID, SUM (SaledMoney) AS TotalSales
FROM Sales S, Dates D
WHERE S.DID=D.DID AND Year BETWEEN 1997 AND 1998
CUBE BY PID, SID, DID
ROLLUP Day, Month, Quarter, Year;
```

- CUBE(TV, No1, 1998)
- CUBE(TV, No1, 1998, Q4)
- CUBE(TV, No1, 1998, 12)



Design Issues



- Fact table in BCNF; dimension tables un-normalized.
 - Dimension tables are small; updates/inserts/deletes are rare. So, anomalies less important than query performance.
- This kind of schema is very common in OLAP applications, and is called a **star schema**; computing the join of all these relations is called a **star join**.



Implementation Issues

- New indexing techniques: Bitmap indexes, Join indexes, array representations, compression, precomputation of aggregations, etc.
- E.g., Bitmap index: **位图索引 hash 簇集 B树索引**.

Bit-vector: 1 bit for each possible value. Many queries can be answered using bit-vector ops!

custid	name	sex	rating	rating
112	Joe	M	3	00100
115	Ram	M	5	00001
119	Sue	F	5	00001
112	Woo	M	4	00010

Bitmap index – index itself is data

Date	Store	State	Class	Sales
3/1/96	32	NY	A	6
3/1/96	36	MA	A	9
3/1/96	38	NY	B	5
3/1/96	41	CT	A	11
3/1/96	43	NY	A	9
3/1/96	46	RI	B	3
3/1/96	47	CT	B	7
3/1/96	49	NY	A	12

Bitmap Index for Sales

8bit	4bit	2bit	1bit
0	1	1	0
1	0	0	1
0	1	0	1
1	0	1	1
1	0	0	1
0	0	1	1
0	1	1	1
1	1	0	0

Bitmap Index for State

AK	AR	CA	CO	CT	MA	NY	RI	...
0	0	0	0	0	0	1	0	
0	0	0	0	0	0	1	0	
0	0	0	0	0	0	1	0	
0	0	0	0	1	0	0	0	
0	0	0	0	0	0	1	0	
0	0	0	0	0	0	0	1	
0	0	0	0	1	0	0	0	
0	0	0	0	0	0	1	0	

要求——对应 适用于不经常改动的表

- Total sales = ? ($4*8+4*4+4*2+6*1=62$)
- How many class A store in NY ? (3)
- Sales of class A store in NY = ? ($2*8+2*4+1*2+1*1=27$)
- How many stores in CT ? (2)
- Join operation (query product list of class A store in NY)

for Class		
A	B	C
1	0	0
1	0	0
0	1	0
1	0	0
1	0	0
0	1	0
0	1	0
1	0	0

Principles of Database Systems, Xu Lizhen

31

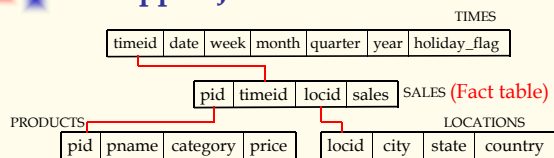
Join Indexes

- Consider the join of Sales, Products, Times, and Locations, possibly with additional selection conditions (e.g., country="USA").
 - A **join index** can be constructed to speed up such joins. The index contains $[s,p,t,l]$ if there are tuples (with sid) s in Sales, p in Products, t in Times and l in Locations that satisfy the join (and selection) conditions.
- **Problem:** Number of join indexes can grow rapidly.
 - A variation addresses this problem: For each column with an additional selection (e.g., country), build an index with $[c,s]$ in it if a dimension table tuple with value c in the selection column joins with a Sales tuple with sid s ; if indexes are bitmaps, called **bitmapped join index**.

Principles of Database Systems, Xu Lizhen

32

Bitmapped Join Index



- Consider a query with conditions price=10 and country="USA". Suppose tuple (with sid) s in Sales joins with a tuple p with price=10 and a tuple l with country="USA". There are two join indexes; one containing $[10,s]$ and the other $[USA,s]$.
- Intersecting these indexes tells us which tuples in Sales are in the join and satisfy the given selection.

Principles of Database Systems, Xu Lizhen

33

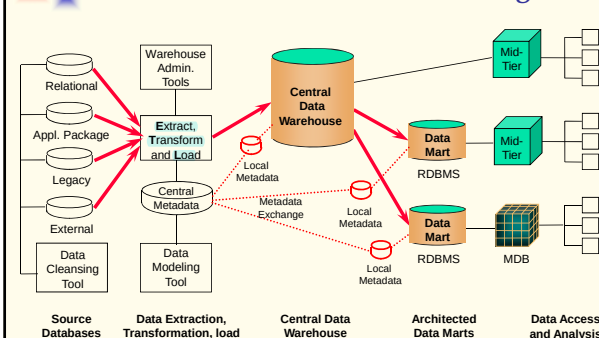
The Procedure of Data Warehouse Engineering

- Requirements Analysis (Project Plan)
- Data Warehouse Architecture Design
- Environments Construction
- Data Warehouse Schema Design
- ETL Processing of Data
- Meta Data Management
- Front Applications Design & Implementation
- Testing
- Running & Maintaining

Principles of Database Systems, Xu Lizhen

34

Data Warehouse Architecture Design



Principles of Database Systems, Xu Lizhen

35

Data Warehouse Schema Design

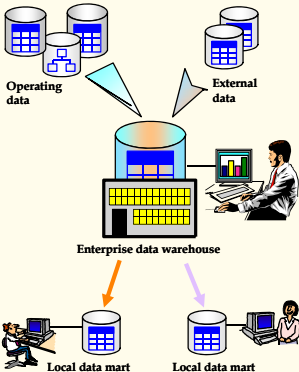
- Top Down
- Bottom Up
- Mixed Method

Principles of Database Systems, Xu Lizhen

36

Top Down

- Construct enterprise data warehouse (EDW)
 - ✓ Common central data schema
 - ✓ One-off data reconstruction
 - ✓ Minimized data redundancy and inconsistency
 - ✓ Detail and historical data; global data exploration
- Construct data marts from EDW
 - ✓ Sub-set in EDW which related with a department
 - ✓ Summarized data
 - ✓ Rely on the availability of data in EDW

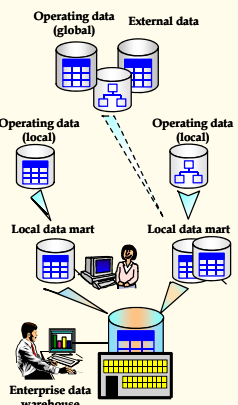


Principles of Database Systems, Xu Lizhen

37

Bottom Up

- Construct data mart of department
 - ✓ Limited in a subject area
 - ✓ Local business requirements are fulfilled rapidly
 - ✓ Department autonomy
 - ✓ A good guidance to the construction of data marts of other departments
 - ✓ Need to do data reconstruction for every department
 - ✓ Have redundancy and inconsistency at a certain extent
 - ✓ A feasible way
- Enlarged to enterprise data warehouse (EDW)
 - ✓ Take the construction of EDW as a long-term target



Principles of Database Systems, Xu Lizhen

38

Summary

- Decision support is an emerging, rapidly growing subarea of databases.
- Involves the creation of large, consolidated data repositories called data warehouses.
- Warehouses exploited using sophisticated analysis techniques: complex SQL queries and OLAP "multidimensional" queries (influenced by both SQL and spreadsheets).
- New techniques for database design, indexing, view maintenance, and interactive querying need to be supported.

Principles of Database Systems, Xu Lizhen

39